

Python source book

All Python used by Agent F (supervised Cursor chat) and Agent H (autonomous Agentic bot). Live place_* is Robinhood MCP, not a side effect of this code. H's standing prompt is markdown: `playbooks/agent_h_autonomous.PROMPT.md` · not included here.

23 files · generated 2026-09-06 00:26 UTC

Language: Python 3 · Letter portrait

Contents

0 · Package marker	pipeline/__init__.py	F + H	4 Pipeline package
1 · Shared loader, journal helpers	pipeline/io_util.py	F + H	39 Paths, rules.json
1 · Shared entries 09:45:15:45; H lease is on origin/main	pipeline/session.py	F + H	91 RTH clock; option
1 · Shared states for Robinhood MCP (no open=true)	pipeline/orders.py	F + H	45 Working-order
1 · Shared executable price; BOD NLV field extract	pipeline/quotes.py	F + H	103 5s underlying
1 · Shared 0.49% planned loss and 0.50% with fees	pipeline/fees.py	F + H	331 Dual fee ceilings:
2 · Agent A crypto drop, inverse-ETF reject, ADV · 2,000,000	pipeline/universe.py	F + H	200 Watchlist extract,
3 · Agent B double-triple / triangle; no 1m/3m/5m	pipeline/patterns.py	F + H	354 Daily-first H&S /
4 · Agent C news/earnings pack; no invented sentiment	pipeline/news.py	F + H	32 Factual RH
5 · Agent D ATM/OTM; 2:3 DTE while overnight off	pipeline/options_structure.py	F + H	167 Long call/put;
5 · Agent D inverse-ETF denylist; H has no equity fallback	pipeline/equity_day_trade.py	F only	246 Long shares only;
6 · Agent I only; signed call +0.40+0.50 / put -0.50-0.40	pipeline/greeks.py	F + H	51 Copy RH Greeks
7 · Agent E equity -20%/+25%; stop first until OCO	pipeline/risk.py	F + H	88 Options -20%/+40%;
8 · Agent G snapshot; h_entry_ready is always false	pipeline/orchestrator.py	F + H	454 Phase 2 read-only
9 · Agent F place-gate: confirm, RTH, 09:45 options, H-owns-RTH	pipeline/execution.py	F (chat)	174 Supervised
10 · CLI data/raw/latest_raw.json and run the orchestrator	scripts/run_phase2_cycle.py	F + H	39 Load
10 · CLI rebuilds the printable source book	scripts/build_python_source_book.py	docs	395 This generator ·
11 · Tests liquidity, signed delta, ATM/OTM, expiration rank	pipeline/tests/test_phase2.py	CI / F	337 Universe,
11 · Tests locked agent_h schema	pipeline/tests/test_orders.py	CI / F	239 Working states and
11 · Tests ceilings	pipeline/tests/test_fees.py	CI / F	93 Dual NLV fee
11 · Tests and flatten window	pipeline/tests/test_session.py	CI / F	16 ET calendar date
11 · Tests selection and Phase 2 snapshots	pipeline/tests/test_equity_day_trade.py	CI / F	451 Long-only equity
11 · Tests	pipeline/tests/test_execution.py	CI / F	147 F place-gate

including 09:45 option lock

```
1  """Phase 2 read-only signal pipeline (Agents A·E + I)."""
2
3  __version__ = "0.2.0"
```

```
1 from __future__ import annotations
2
3 import json
4 from datetime import datetime, timezone
5 from pathlib import Path
6 from typing import Any
7
8
9 ROOT = Path(__file__).resolve().parents[1]
10 SIGNALS = ROOT / "signals"
11 JOURNAL = ROOT / "journal"
12 DATA_RAW = ROOT / "data" / "raw"
13 CONFIG = ROOT / "config" / "rules.json"
14
15
16 def utc_now_iso() -> str:
17     return datetime.now(timezone.utc).replace(microsecond=0).isoformat()
18
19
20 def load_rules() -> dict[str, Any]:
21     return json.loads(CONFIG.read_text(encoding="utf-8"))
22
23
24 def write_json(path: Path, payload: Any) -> Path:
25     path.parent.mkdir(parents=True, exist_ok=True)
26     path.write_text(json.dumps(payload, indent=2, sort_keys=False) + "\n", encoding="utf-8")
27     return path
28
29
30 def append_jsonl(path: Path, row: dict[str, Any]) -> Path:
31     path.parent.mkdir(parents=True, exist_ok=True)
32     with path.open("a", encoding="utf-8") as f:
33         f.write(json.dumps(row, sort_keys=False) + "\n")
34     return path
35
36
37 def read_json(path: Path) -> Any:
38     return json.loads(path.read_text(encoding="utf-8"))
```

```
1 from __future__ import annotations
2
3 from datetime import date, datetime, time
4 from zoneinfo import ZoneInfo
5 from typing import Any
6
7 ET = ZoneInfo("America/New_York")
8 RTH_START = time(9, 30)
9 RTH_END = time(16, 0)
10 NO_NEW_OPTION_ENTRIES_BEFORE = time(9, 45)
11 NO_NEW_ENTRIES_AFTER = time(15, 45)
12
13
14 def now_et(now: datetime | None = None) -> datetime:
15     if now is None:
16         return datetime.now(ET)
17     if now.tzinfo is None:
18         return now.replace(tzinfo=ET)
19     return now.astimezone(ET)
20
21
22 def today_et(now: datetime | None = None) -> date:
23     """Calendar date in America/New_York. Do not use UTC date.today() for DTE or journals."""
24     return now_et(now).date()
25
26
27 def is_weekday(now: datetime | None = None) -> bool:
28     return now_et(now).weekday() < 5
29
30
31 def is_rth(now: datetime | None = None) -> bool:
32     """Mon-Fri 09:30 inclusive through 16:00 exclusive America/New_York."""
33     dt = now_et(now)
34     if not is_weekday(dt):
35         return False
36     t = dt.time()
37     return RTH_START <= t < RTH_END
38
39
40 def entries_open(now: datetime | None = None) -> bool:
41     """Equity / generic new entries: RTH before 15:45 ET."""
42     dt = now_et(now)
43     if not is_rth(dt):
44         return False
45     return dt.time() < NO_NEW_ENTRIES_AFTER
46
47
48 def option_entries_open(now: datetime | None = None) -> bool:
49     """New option entries: RTH from 09:45 inclusive through 15:45 exclusive ET."""
50     dt = now_et(now)
51     if not entries_open(dt):
52         return False
53     return dt.time() >= NO_NEW_OPTION_ENTRIES_BEFORE
54
55
56 def flatten_window(now: datetime | None = None) -> bool:
57     """Still RTH, but new entries are closed (15:45-16:00 ET)."""
58     dt = now_et(now)
59     if not is_rth(dt):
60         return False
61     return dt.time() >= NO_NEW_ENTRIES_AFTER
62
63
64 def session_gate(now: datetime | None = None) -> dict[str, Any]:
65     dt = now_et(now)
66     rth = is_rth(dt)
67     open_for_entry = entries_open(dt)
68     option_open = option_entries_open(dt)
69     reason = None
70     if not rth:
71         reason = "outside_rth"
72     elif not open_for_entry:
73         reason = "no_new_entries_after_1545"
74     option_reason = None
```

```
75     if not rth:
76         option_reason = "outside_rth"
77     elif dt.time() < NO_NEW_OPTION_ENTRIES_BEFORE:
78         option_reason = "no_new_option_entries_before_0945"
79     elif not open_for_entry:
80         option_reason = "no_new_entries_after_1545"
81     return {
82         "timezone": "America/New_York",
83         "now_et": dt.isoformat(),
84         "is_rth": rth,
85         "entries_open": open_for_entry,
86         "option_entries_open": option_open,
87         "flatten_window": flatten_window(dt),
88         "reason": reason,
89         "option_reason": option_reason,
90     }
```

```
1 from __future__ import annotations
2
3 from typing import Any, Iterable
4
5 # Robinhood MCP has no `open=true` filter. Working tickets are these states.
6 OPTION_WORKING_STATES = frozenset(
7     {"queued", "confirmed", "partially_filled", "pending_cancelled"}
8 )
9 EQUITY_WORKING_STATES = frozenset(
10     {"new", "queued", "confirmed", "unconfirmed", "partially_filled"}
11 )
12
13
14 def normalize_state(value: Any) -> str:
15     return str(value or "").strip().lower()
16
17
18 def is_working_option_state(state: Any) -> bool:
19     return normalize_state(state) in OPTION_WORKING_STATES
20
21
22 def is_working_equity_state(state: Any) -> bool:
23     return normalize_state(state) in EQUITY_WORKING_STATES
24
25
26 def working_orders(
27     option_orders: Iterable[dict[str, Any]] | None = None,
28     equity_orders: Iterable[dict[str, Any]] | None = None,
29 ) -> list[dict[str, Any]]:
30     found: list[dict[str, Any]] = []
31     for row in option_orders or []:
32         if is_working_option_state(row.get("state") or row.get("status")):
33             found.append({"asset_class": "option", **row})
34     for row in equity_orders or []:
35         if is_working_equity_state(row.get("state") or row.get("status")):
36             found.append({"asset_class": "equity", **row})
37     return found
38
39
40 def has_working_orders(
41     option_orders: Iterable[dict[str, Any]] | None = None,
42     equity_orders: Iterable[dict[str, Any]] | None = None,
43 ) -> bool:
44     return bool(working_orders(option_orders, equity_orders))
```

```
1 from __future__ import annotations
2
3 from datetime import datetime, timedelta, timezone
4 from typing import Any
5
6
7 UNDERLYING_MAX_AGE_SECONDS = 5
8 BOD_NLV_FIELD_CANDIDATES = (
9     "start_of_day_equity",
10    "beginning_of_day_equity",
11    "bod_equity",
12    "bod_nlv",
13    "equity_start_of_day",
14    "start_of_day_portfolio_value",
15    "beginning_of_day_portfolio_value",
16    "last_core_portfolio_equity",
17    "last_core_equity",
18 )
19
20
21 def _parse_ts(value: Any) -> datetime | None:
22     if value in (None, ""):
23         return None
24     if isinstance(value, datetime):
25         if value.tzinfo is None:
26             return value.replace(tzinfo=timezone.utc)
27         return value
28     text = str(value).strip()
29     if not text:
30         return None
31     if text.endswith("Z"):
32         text = text[:-1] + "+00:00"
33     try:
34         parsed = datetime.fromisoformat(text)
35     except ValueError:
36         return None
37     if parsed.tzinfo is None:
38         parsed = parsed.replace(tzinfo=timezone.utc)
39     return parsed
40
41
42 def _positive_money(value: Any) -> float | None:
43     if value in (None, ""):
44         return None
45     try:
46         amount = float(str(value).replace("$", "").replace(",", ""))
47     except (TypeError, ValueError):
48         return None
49     if amount != amount or amount <= 0:
50         return None
51     return amount
52
53
54 def executable_underlying_price(
55     quote: dict[str, Any] | None,
56     *,
57     now: datetime | None = None,
58     max_age_seconds: int = UNDERLYING_MAX_AGE_SECONDS,
59 ) -> tuple[float | None, str | None]:
60     """Regular-session executable price for the live breakout trigger."""
61     if not isinstance(quote, dict):
62         return None, "underlying_quote_missing"
63     bid = _positive_money(quote.get("bid_price", quote.get("bid")))
64     ask = _positive_money(quote.get("ask_price", quote.get("ask")))
65     if bid is None or ask is None:
66         return None, "underlying_bid_ask_missing"
67     if bid > ask:
68         return None, "underlying_bid_above_ask"
69     ts = _parse_ts(
70         quote.get("updated_at")
71         or quote.get("updated_at_utc")
72         or quote.get("ask_time")
73         or quote.get("bid_time")
74         or quote.get("last_trade_time")
75     )
```

```
75     )
76     now = now or datetime.now(timezone.utc)
77     if ts is None:
78         return None, "underlying_quote_timestamp_missing"
79     if now - ts > timedelta(seconds=max_age_seconds):
80         return None, "underlying_quote_stale"
81     last = _positive_money(quote.get("last_trade_price", quote.get("last_price", quote.get("last"))))
82     if last is not None and bid <= last <= ask:
83         return last, None
84     return (bid + ask) / 2.0, None
85
86
87 def extract_bod_nlv(portfolio: dict[str, Any] | None) -> tuple[float | None, str | None]:
88     """Return a broker beginning-of-day NLV if a known field is present. Never invent it."""
89     if not isinstance(portfolio, dict):
90         return None, None
91     mappings = [portfolio]
92     nested = portfolio.get("equity")
93     if isinstance(nested, dict):
94         mappings.append(nested)
95     for mapping in mappings:
96         for key in BOD_NLV_FIELD_CANDIDATES:
97             if key in mapping:
98                 amount = _positive_money(mapping.get(key))
99                 if amount is None:
100                     return None, key
101                 return amount, key
102     return None, None
```

```
1 from __future__ import annotations
2
3 from typing import Any
4
5 # Named leaf fees. Do not include totals or group subtotals here.
6 COMPONENT_KEYS = frozenset(
7     {
8         "commission",
9         "commissions",
10        "regulatory_fee",
11        "regulatory_fees",
12        "regulatory",
13        "sec_fee",
14        "sec_fees",
15        "taf_fee",
16        "taf_fees",
17        "option_regulatory_fee",
18        "orf",
19        "orf_fee",
20        "contract_fee",
21        "contract_fees",
22        "per_contract_fee",
23        "exchange_fee",
24        "exchange_fees",
25        "clearing_fee",
26        "clearing_fees",
27    }
28 )
29
30 # Group keys that likely already include one or more COMPONENT_KEYS.
31 COMPONENT_FAMILIES: tuple[frozenset[str], ...] = (
32     frozenset({"commission", "commissions"}),
33     frozenset(
34         {
35             "regulatory_fee",
36             "regulatory_fees",
37             "regulatory",
38             "sec_fee",
39             "sec_fees",
40             "taf_fee",
41             "taf_fees",
42             "option_regulatory_fee",
43             "orf",
44             "orf_fee",
45         }
46     ),
47     frozenset({"contract_fee", "contract_fees", "per_contract_fee"}),
48     frozenset({"exchange_fee", "exchange_fees"}),
49     frozenset({"clearing_fee", "clearing_fees"}),
50 )
51
52 # Alternate totals / rolled-up fees that must not be added to their parts.
53 SUBTOTAL_KEYS = frozenset(
54     {"fee", "estimated_fee", "estimated_fees", "fee_total", "fees_total"}
55 )
56
57 PLANNED_LOSS_CEILING_WITH_QUOTED_FEE = 0.005
58 PLANNED_LOSS_CEILING_IF_FEE_UNAVAILABLE_OR_ZERO = 0.0049
59
60
61 def parse_money(value: Any) -> tuple[float | None, bool]:
62     """Return (amount, readable). readable is False when the field cannot be used."""
63     if value is None:
64         return None, True
65     if isinstance(value, bool):
66         return None, False
67     if isinstance(value, (int, float)):
68         if value != value or value in (float("inf"), float("-inf")): # NaN / inf
69             return None, False
70         return float(value), True
71     if isinstance(value, str):
72         text = value.strip().replace("$", "").replace(",", "")
73         if text == "":
74             return None, True
```

```
75         try:
76             amount = float(text)
77         except ValueError:
78             return None, False
79         if amount != amount or amount in (float("inf"), float("-inf")):
80             return None, False
81         return amount, True
82     return None, False
83
84
85 def _unavailable(reason: str) -> dict[str, Any]:
86     return {
87         "fee_status": "unavailable",
88         "entry_fee": None,
89         "journal": "fee_unavailable",
90         "source": reason,
91         "apply_049_ceiling": True,
92         "estimated_exit_fee": None,
93         "estimated_round_trip_fees": None,
94     }
95
96
97 def _explicit_zero(source: str) -> dict[str, Any]:
98     return {
99         "fee_status": "explicit_zero",
100         "entry_fee": 0.0,
101         "journal": "fee_explicit_zero",
102         "source": source,
103         "apply_049_ceiling": True,
104         "estimated_exit_fee": None,
105         "estimated_round_trip_fees": None,
106     }
107
108
109 def _quoted(entry_fee: float, source: str) -> dict[str, Any]:
110     return {
111         "fee_status": "quoted",
112         "entry_fee": entry_fee,
113         "journal": source,
114         "source": source,
115         "apply_049_ceiling": True,
116         "estimated_exit_fee": 2.0 * entry_fee,
117         "estimated_round_trip_fees": 3.0 * entry_fee,
118     }
119
120
121 def _conflict(source: str) -> dict[str, Any]:
122     return {
123         "fee_status": "ambiguous",
124         "entry_fee": None,
125         "journal": "fee_conflict",
126         "source": source,
127         "apply_049_ceiling": True,
128         "estimated_exit_fee": None,
129         "estimated_round_trip_fees": None,
130     }
131
132
133 def _fee_mappings(review: Any) -> list[dict[str, Any]]:
134     if not isinstance(review, dict):
135         return []
136     mappings = [review]
137     nested = review.get("fees")
138     if isinstance(nested, dict):
139         mappings.append(nested)
140     return mappings
141
142
143 def _list_components(review: Any) -> list[tuple[str, Any]]:
144     rows: list[tuple[str, Any]] = []
145     if not isinstance(review, dict):
146         return rows
147     nested = review.get("fees")
148     if not isinstance(nested, list):
```

```
149         return rows
150     for item in nested:
151         if not isinstance(item, dict):
152             continue
153         name = str(
154             item.get("type")
155             or item.get("name")
156             or item.get("label")
157             or item.get("key")
158             or ""
159         ).strip().lower()
160         amount = item.get("amount", item.get("fee", item.get("value")))
161         if name:
162             rows.append((name, amount))
163     return rows
164
165
166 def _first_total(mappings: list[dict[str, Any]]) -> tuple[Any, bool]:
167     """Return (raw_value, present). Prefer total_fee over aliases."""
168     present = False
169     raw: Any = None
170     for mapping in mappings:
171         if "total_fee" in mapping:
172             return mapping["total_fee"], True
173         if "total_fees" in mapping and not present:
174             raw = mapping["total_fees"]
175             present = True
176         if mapping.get("total") is not None and "commission" in mapping and not present:
177             # Nested RH-style {total, commission, ...}
178             raw = mapping["total"]
179             present = True
180     return raw, present
181
182
183 def _component_values(mappings: list[dict[str, Any]], extra: list[tuple[str, Any]]) -> dict[str, Any]:
184     values: dict[str, Any] = {}
185     for mapping in mappings:
186         for key in COMPONENT_KEYS:
187             if key in mapping:
188                 values[key] = mapping[key]
189     for name, amount in extra:
190         if name in COMPONENT_KEYS:
191             values[name] = amount
192         elif name in {"sec", "taf", "orf"}:
193             values[f"{name}_fee"] = amount
194     return values
195
196
197 def _subtotal_values(mappings: list[dict[str, Any]]) -> dict[str, Any]:
198     values: dict[str, Any] = {}
199     for mapping in mappings:
200         for key in SUBTOTAL_KEYS:
201             if key in mapping:
202                 values[key] = mapping[key]
203     return values
204
205
206 def _parsed_items(raw_items: dict[str, Any]) -> tuple[dict[str, float], str | None]:
207     parsed: dict[str, float] = {}
208     for key, raw in raw_items.items():
209         amount, readable = parse_money(raw)
210         if not readable:
211             return {}, "unreadable"
212         if amount is None:
213             continue
214         if amount < 0:
215             return {}, "negative"
216         parsed[key] = amount
217     return parsed, None
218
219
220 def _components_overlap(keys: set[str]) -> bool:
221     for family in COMPONENT_FAMILIES:
222         hit = keys & family
```

```
223     if len(hit) <= 1:
224         continue
225     group = hit & {
226         "commission",
227         "commissions",
228         "regulatory_fee",
229         "regulatory_fees",
230         "regulatory",
231         "contract_fee",
232         "contract_fees",
233         "exchange_fee",
234         "exchange_fees",
235         "clearing_fee",
236         "clearing_fees",
237     }
238     parts = hit - group
239     if group and parts:
240         return True
241     if len(hit & {"commission", "commissions"}) == 2:
242         return True
243     if len(hit & {"contract_fee", "contract_fees", "per_contract_fee"}) > 1 and not parts:
244         # two names for the same contract fee
245         if len(hit & {"contract_fee", "contract_fees"}) == 2:
246             return True
247     return False
248
249
250 def classify_review_fees(review: Any) -> dict[str, Any]:
251     """Parse review_option_order fees. Never trust $0.00 total with positive parts."""
252     if review is None or not isinstance(review, dict):
253         return _unavailable("review_unreadable")
254
255     mappings = _fee_mappings(review)
256     raw_total, total_present = _first_total(mappings)
257     components_raw = _component_values(mappings, _list_components(review))
258     subtotals_raw = _subtotal_values(mappings)
259
260     if total_present:
261         total, readable = parse_money(raw_total)
262         if not readable or total is None or total < 0:
263             return _unavailable("total_fee_unreadable")
264
265         components, error = _parsed_items(components_raw)
266         if error:
267             return _unavailable(f"component_{error}")
268
269         if total > 0:
270             return _quoted(total, "total_fee")
271
272         positive = {key: amount for key, amount in components.items() if amount > 0}
273         if positive:
274             return _conflict(
275                 "zero_total_plus_positive_component:" + ", ".join(sorted(positive))
276             )
277         return _explicit_zero("total_fee_and_components_zero_or_absent")
278
279     components, error = _parsed_items(components_raw)
280     if error:
281         return _unavailable(f"component_{error}")
282     subtotals, error = _parsed_items(subtotals_raw)
283     if error:
284         return _unavailable(f"subtotal_{error}")
285
286     if subtotals and components:
287         return _unavailable("subtotal_plus_parts")
288     if _components_overlap(set(components)):
289         return _unavailable("overlapping_components")
290
291     if subtotals:
292         values = list(subtotals.values())
293         if len(set(values)) > 1:
294             return _unavailable("duplicated_subtotals")
295         amount = values[0]
296         source = next(iter(subtotals))
```

```
297         if amount > 0:
298             return _quoted(amount, source)
299         return _explicit_zero(source)
300
301     if components:
302         amount = sum(components.values())
303         source = "components:" + ", ".join(sorted(components))
304         if amount > 0:
305             return _quoted(amount, source)
306         return _explicit_zero(source)
307
308     return _unavailable("no_fee_fields")
309
310
311 def fee_aware_planned_loss_ok(
312     *,
313     planned_loss: float,
314     current_nlv: float,
315     classification: dict[str, Any],
316 ) -> bool:
317     """Both ceilings apply on every trade. Missing fees count as $0 in the 0.50% sum."""
318     if current_nlv <= 0 or planned_loss < 0:
319         return False
320     if planned_loss > PLANNED_LOSS_CEILING_IF_FEE_UNAVAILABLE_OR_ZERO * current_nlv + 1e-12:
321         return False
322     round_trip = classification.get("estimated_round_trip_fees")
323     if not isinstance(round_trip, (int, float)):
324         round_trip = 0.0
325     if float(round_trip) < 0:
326         return False
327     return (
328         planned_loss + float(round_trip)
329         <= PLANNED_LOSS_CEILING_WITH_QUOTED_FEE * current_nlv + 1e-12
330     )
```

```

1  from __future__ import annotations
2
3  from typing import Any
4
5  from pipeline.equity_day_trade import is_inverse_etf
6
7
8  CRYPTO_OBJECT_TYPES = {"currency_pair", "tokenized_stock"}
9  EQUITY_OBJECT_TYPES = {"instrument"}
10 INDEX_OBJECT_TYPES = {"index"}
11 OPTION_OBJECT_TYPES = {"option_strategy", "option"}
12
13
14 def _normalize_symbol(value: str | None) -> str | None:
15     if not value:
16         return None
17     return value.strip().upper()
18
19
20 def extract_watchlist_symbols(
21     watchlists: list[dict[str, Any]],
22     items_by_list: dict[str, list[dict[str, Any]]],
23     option_watchlist_items: list[dict[str, Any]] | None,
24     *,
25     include_crypto: bool = False,
26     include_options_watchlist: bool = True,
27 ) -> dict[str, Any]:
28     """Agent A: union symbols from all lists; exclude crypto unless enabled."""
29     equities: dict[str, dict[str, Any]] = {}
30     indexes: dict[str, dict[str, Any]] = {}
31     skipped_crypto: list[str] = []
32     sources: dict[str, list[str]] = {}
33
34     for wl in watchlists:
35         list_id = wl.get("id")
36         name = wl.get("display_name") or list_id
37         for item in items_by_list.get(list_id, []):
38             obj_type = (item.get("object_type") or item.get("type") or "").lower()
39             symbol = _normalize_symbol(
40                 item.get("symbol")
41                 or item.get("display_symbol")
42                 or item.get("equity_symbol")
43                 or item.get("chain_symbol")
44             )
45             if not symbol:
46                 continue
47             sources.setdefault(symbol, [])
48             if name not in sources[symbol]:
49                 sources[symbol].append(str(name))
50
51     if obj_type in CRYPTO_OBJECT_TYPES or (not obj_type and "-" in symbol and symbol.endswith("USD")):
52         if not include_crypto:
53             skipped_crypto.append(symbol)
54             continue
55         if obj_type in INDEX_OBJECT_TYPES:
56             indexes[symbol] = {"symbol": symbol, "object_type": "index", "sources": sources[symbol]}
57         elif obj_type in OPTION_OBJECT_TYPES:
58             # Options watchlist handled separately; still capture underlying if present.
59             und = _normalize_symbol(item.get("chain_symbol") or item.get("underlying_symbol"))
60             if und:
61                 equities.setdefault(und, {"symbol": und, "object_type": "instrument", "sources": sources.get(und, [str(name)])})
62             else:
63                 # Default: treat as equity/ETF instrument.
64                 equities[symbol] = {"symbol": symbol, "object_type": "instrument", "sources": sources[symbol]}
65
66     option_underlyings: list[str] = []
67     if include_options_watchlist and option_watchlist_items:
68         for item in option_watchlist_items:
69             und = _normalize_symbol(
70                 item.get("chain_symbol")
71                 or item.get("underlying_symbol")
72                 or item.get("symbol")
73             )

```

```

74         if und:
75             option_underlyings.append(und)
76             equities.setdefault(
77                 und,
78                 {"symbol": und, "object_type": "instrument", "sources": ["Options Watchlist"]},
79             )
80
81     return {
82         "equities": sorted(equities.values(), key=lambda x: x["symbol"]),
83         "indexes": sorted(indexes.values(), key=lambda x: x["symbol"]),
84         "option_watchlist_underlyings": sorted(set(option_underlyings)),
85         "skipped_crypto": sorted(set(skipped_crypto)),
86         "equity_symbols": sorted(equities.keys()),
87         "index_symbols": sorted(indexes.keys()),
88     }
89
90
91 def apply_liquidity_filter(
92     symbols: list[str],
93     fundamentals_by_symbol: dict[str, dict[str, Any]],
94     *,
95     min_average_volume: float,
96 ) -> dict[str, Any]:
97     """Filter equities by average volume from fundamentals payloads."""
98     passed: list[dict[str, Any]] = []
99     rejected: list[dict[str, Any]] = []
100
101     for symbol in symbols:
102         fund = fundamentals_by_symbol.get(symbol) or {}
103         if is_inverse_etf(symbol, fund):
104             rejected.append({"symbol": symbol, "reason": "inverse_etf"})
105             continue
106         # RH fundamentals field names can vary; accept common keys only if present.
107         avg_vol = None
108         for key in (
109             "average_volume",
110             "average_volume_2_weeks",
111             "avg_volume",
112             "volume",
113         ):
114             if key in fund and fund[key] not in (None, ""):
115                 try:
116                     avg_vol = float(fund[key])
117                     break
118                 except (TypeError, ValueError):
119                     continue
120             if avg_vol is None:
121                 rejected.append({"symbol": symbol, "reason": "missing_average_volume", "average_volume": None})
122                 continue
123             if avg_vol < min_average_volume:
124                 rejected.append(
125                     {
126                         "symbol": symbol,
127                         "reason": "below_min_average_volume",
128                         "average_volume": avg_vol,
129                         "min_average_volume": min_average_volume,
130                     }
131                 )
132                 continue
133             passed.append({"symbol": symbol, "average_volume": avg_vol})
134
135     return {
136         "passed": passed,
137         "rejected": rejected,
138         "passed_symbols": [p["symbol"] for p in passed],
139     }
140
141
142 def option_quote_liquid(
143     quote: dict[str, Any],
144     *,
145     max_spread_pct_of_price: float,
146     preferred_spread_pct_of_price: float = 0.05,
147     reject_one_sided: bool = True,

```



```
148 ) -> tuple[bool, str | None, dict[str, Any]]:
149     """Return (ok, reason, metrics) for an option quote liquidity gate.
150
151     Spread is measured as (ask · bid) / mid. Prefer · 5% of price; reject above 10%.
152     There is no absolute-dollar override.
153     """
154     if not quote:
155         return False, "missing_bid_ask", {}
156
157     def _px(*keys: str) -> float | None:
158         for key in keys:
159             if key in quote and quote[key] not in (None, ""):
160                 try:
161                     return float(quote[key])
162                 except (TypeError, ValueError):
163                     continue
164         return None
165
166     bid = _px("bid_price", "bid", "bid_last")
167     ask = _px("ask_price", "ask", "ask_last")
168     if bid is None or ask is None:
169         return False, "missing_bid_ask", {"bid": bid, "ask": ask}
170
171     if reject_one_sided and (bid <= 0 or ask <= 0):
172         return False, "one_sided_or_missing_quote", {"bid": bid, "ask": ask}
173
174     mid = (bid + ask) / 2.0
175     spread = ask - bid
176     spread_pct = (spread / mid) if mid > 0 else None
177     metrics: dict[str, Any] = {
178         "bid": bid,
179         "ask": ask,
180         "mid": mid,
181         "spread": spread,
182         "spread_pct_of_price": spread_pct,
183         "spread_pct_of_mid": spread_pct,
184         "preferred_spread_pct_of_price": preferred_spread_pct_of_price,
185         "max_spread_pct_of_price": max_spread_pct_of_price,
186     }
187
188     if mid <= 0:
189         return False, "non_positive_mid", metrics
190     if spread_pct is None:
191         return False, "spread_too_wide", metrics
192     if spread_pct <= preferred_spread_pct_of_price:
193         metrics["spread_quality"] = "preferred"
194         return True, None, metrics
195     if spread_pct <= max_spread_pct_of_price:
196         metrics["spread_quality"] = "acceptable"
197         return True, None, metrics
198     metrics["spread_quality"] = "too_wide"
199     return False, "spread_too_wide", metrics
```

```

1 from __future__ import annotations
2
3 from typing import Any
4
5 import numpy as np
6
7
8 def _local_extrema(prices: np.ndarray, order: int = 3) -> tuple[list[int], list[int]]:
9     """Simple peak/trough detection with `order` bars on each side."""
10    peaks: list[int] = []
11    troughs: list[int] = []
12    n = len(prices)
13    for i in range(order, n - order):
14        window = prices[i - order : i + order + 1]
15        if prices[i] == window.max() and np.sum(window == prices[i]) == 1:
16            peaks.append(i)
17        if prices[i] == window.min() and np.sum(window == prices[i]) == 1:
18            troughs.append(i)
19    return peaks, troughs
20
21
22 def _nearly_equal(a: float, b: float, tol_pct: float = 0.015) -> bool:
23     base = max(abs(a), abs(b), 1e-9)
24     return abs(a - b) / base <= tol_pct
25
26
27 PATTERN_PRIORITY = (
28     "inverse_head_and_shoulders",
29     "head_and_shoulders",
30     "double_bottom",
31     "double_top",
32     "triple_bottom",
33     "triple_top",
34     "ascending_triangle",
35     "descending_triangle",
36 )
37 PATTERN_FAMILY = {
38     "inverse_head_and_shoulders": 0,
39     "head_and_shoulders": 0,
40     "double_bottom": 1,
41     "double_top": 1,
42     "triple_bottom": 1,
43     "triple_top": 1,
44     "ascending_triangle": 2,
45     "descending_triangle": 2,
46 }
47
48 HEAD_PROMINENCE_PCT = 0.015
49 MIN_PIVOT_SEPARATION_BARS = 3
50 MAX_PATTERN_BARS = {
51     "day": 60,
52     "daily": 60,
53     "hour": 40,
54     "10minute": 30,
55 }
56 TRIANGLE_TOUCH_PCT = 0.005
57 TRIANGLE_MIN_TOUCHES_PER_SIDE = 2
58
59
60 def _max_pattern_bars(timeframe: str) -> int:
61     return MAX_PATTERN_BARS.get(timeframe, 60)
62
63
64 def _pivots_separated(indices: list[int], *, min_gap: int = MIN_PIVOT_SEPARATION_BARS) -> bool:
65     ordered = sorted(indices)
66     return all(b - a >= min_gap for a, b in zip(ordered, ordered[1:]))
67
68
69 def _count_line_touches(prices: np.ndarray, slope: float, intercept: float, *, tol_pct: float) -> int:
70     touches = 0
71     for i, price in enumerate(prices):
72         fitted = intercept + slope * float(i)
73         base = max(abs(fitted), abs(float(price)), 1e-9)
74         if abs(float(price) - fitted) / base <= tol_pct:

```

```

75         touches += 1
76     return touches
77
78
79 def _last_touch_offset(prices: np.ndarray, slope: float, intercept: float, *, tol_pct: float) -> int |
None:
80     last: int | None = None
81     for i, price in enumerate(prices):
82         fitted = intercept + slope * float(i)
83         base = max(abs(fitted), abs(float(price)), 1e-9)
84         if abs(float(price) - fitted) / base <= tol_pct:
85             last = i
86     return last
87
88
89 def _span_ok(indices: list[int], *, max_span: int) -> bool:
90     return (max(indices) - min(indices)) <= max_span
91
92
93 def _last_pivot(hit: dict[str, Any]) -> int:
94     if hit.get("last_pivot") is not None:
95         return int(hit["last_pivot"])
96     indices = hit.get("indices") or [0]
97     return int(max(indices))
98
99
100 def rank_daily_setups(hits: list[dict[str, Any]]) -> list[dict[str, Any]]:
101     """One deterministic winner among overlapping daily setups. Neutral triangles never rank."""
102     daily = [
103         hit
104         for hit in hits
105     if hit.get("timeframe") in ("day", "daily") and hit.get("bias") not in (None, "neutral", "none")
106     ]
107     priority = {name: i for i, name in enumerate(PATTERN_PRIORITY)}
108
109     def sort_key(hit: dict[str, Any]) -> tuple[int, int, float, int]:
110         name = str(hit.get("pattern"))
111         family = PATTERN_FAMILY.get(name, 99)
112         prominence = float(hit.get("prominence") or 0.0)
113         # Family first so a triangle window-end cannot leapfrog H&S or double/triple.
114         return (family, -_last_pivot(hit), -prominence, priority.get(name, 99))
115
116     return sorted(daily, key=sort_key)
117
118
119 def detect_patterns(ohlc: list[dict[str, Any]], *, timeframe: str) -> list[dict[str, Any]]:
120     """
121     Deterministic pattern heuristics on close prices.
122     Returns pattern hits with indices; empty if insufficient bars.
123     """
124     if len(ohlc) < 30:
125         return []
126
127     closes = np.array([float(b["close"]) for b in ohlc], dtype=float)
128     highs = np.array([float(b.get("high", b["close"])) for b in ohlc], dtype=float)
129     lows = np.array([float(b.get("low", b["close"])) for b in ohlc], dtype=float)
130     peaks, troughs = _local_extrema(closes, order=3)
131     hits: list[dict[str, Any]] = []
132     max_span = _max_patternBars(timeframe)
133
134     # Double / triple tops
135     if len(peaks) >= 2:
136         p1, p2 = peaks[-2], peaks[-1]
137         if (
138             _nearly_equal(closes[p1], closes[p2])
139             and _pivots_separated([p1, p2])
140             and _span_ok([p1, p2], max_span=max_span)
141         ):
142             neck = float(closes[p1:p2].min()) if p2 > p1 else float(closes[p2])
143             prominence = abs(float(closes[p2]) - neck) / max(abs(neck), 1e-9)
144             hits.append(
145                 {
146                     "pattern": "double_top",
147                     "timeframe": timeframe,

```

```
148         "indices": [p1, p2],
149         "last_pivot": p2,
150         "prices": [float(closes[p1]), float(closes[p2])],
151         "neckline": neck,
152         "prominence": prominence,
153         "bias": "bearish",
154     }
155 )
156 if len(peaks) >= 3:
157     p1, p2, p3 = peaks[-3], peaks[-2], peaks[-1]
158     if (
159         _nearly_equal(closes[p1], closes[p2])
160         and _nearly_equal(closes[p2], closes[p3])
161         and _pivots_separated([p1, p2, p3])
162         and _span_ok([p1, p3], max_span=max_span)
163     ):
164         neck = float(closes[p1:p3].min())
165         prominence = abs(float(closes[p3]) - neck) / max(abs(neck), 1e-9)
166         hits.append(
167             {
168                 "pattern": "triple_top",
169                 "timeframe": timeframe,
170                 "indices": [p1, p2, p3],
171                 "last_pivot": p3,
172                 "prices": [float(closes[p1]), float(closes[p2]), float(closes[p3])],
173                 "neckline": neck,
174                 "prominence": prominence,
175                 "bias": "bearish",
176             }
177         )
178
179 # Double / triple bottoms
180 if len(troughs) >= 2:
181     t1, t2 = troughs[-2], troughs[-1]
182     if (
183         _nearly_equal(closes[t1], closes[t2])
184         and _pivots_separated([t1, t2])
185         and _span_ok([t1, t2], max_span=max_span)
186     ):
187         neck = float(closes[t1:t2].max()) if t2 > t1 else float(closes[t2])
188         prominence = abs(neck - float(closes[t2])) / max(abs(neck), 1e-9)
189         hits.append(
190             {
191                 "pattern": "double_bottom",
192                 "timeframe": timeframe,
193                 "indices": [t1, t2],
194                 "last_pivot": t2,
195                 "prices": [float(closes[t1]), float(closes[t2])],
196                 "neckline": neck,
197                 "prominence": prominence,
198                 "bias": "bullish",
199             }
200         )
201 if len(troughs) >= 3:
202     t1, t2, t3 = troughs[-3], troughs[-2], troughs[-1]
203     if (
204         _nearly_equal(closes[t1], closes[t2])
205         and _nearly_equal(closes[t2], closes[t3])
206         and _pivots_separated([t1, t2, t3])
207         and _span_ok([t1, t3], max_span=max_span)
208     ):
209         neck = float(closes[t1:t3].max())
210         prominence = abs(neck - float(closes[t3])) / max(abs(neck), 1e-9)
211         hits.append(
212             {
213                 "pattern": "triple_bottom",
214                 "timeframe": timeframe,
215                 "indices": [t1, t2, t3],
216                 "last_pivot": t3,
217                 "prices": [float(closes[t1]), float(closes[t2]), float(closes[t3])],
218                 "neckline": neck,
219                 "prominence": prominence,
220                 "bias": "bullish",
221             }
222         )
```

```

222         )
223
224     # Head and shoulders / inverse: time-ordered LS · head · RS, intervening opposite pivots.
225     if len(peaks) >= 3:
226         l, h, r = peaks[-3], peaks[-2], peaks[-1]
227         left_troughs = [t for t in troughs if l < t < h]
228         right_troughs = [t for t in troughs if h < t < r]
229         head_vs_left = (closes[h] - closes[l]) / max(abs(closes[l]), 1e-9)
230         head_vs_right = (closes[h] - closes[r]) / max(abs(closes[r]), 1e-9)
231         if (
232             l < h < r
233             and (r - l) <= max_span
234             and _pivots_separated([l, h, r])
235             and left_troughs
236             and right_troughs
237             and closes[h] > closes[l]
238             and closes[h] > closes[r]
239             and head_vs_left >= HEAD_PROMINENCE_PCT
240             and head_vs_right >= HEAD_PROMINENCE_PCT
241             and _nearly_equal(closes[l], closes[r], tol_pct=0.025)
242         ):
243             neck = float((closes[left_troughs[-1]] + closes[right_troughs[0]]) / 2.0)
244             hits.append(
245                 {
246                     "pattern": "head_and_shoulders",
247                     "timeframe": timeframe,
248                     "indices": [l, h, r],
249                     "last_pivot": r,
250                     "prices": [float(closes[l]), float(closes[h]), float(closes[r])],
251                     "neckline": neck,
252                     "prominence": float(min(head_vs_left, head_vs_right)),
253                     "bias": "bearish",
254                 }
255             )
256     if len(troughs) >= 3:
257         l, h, r = troughs[-3], troughs[-2], troughs[-1]
258         left_peaks = [p for p in peaks if l < p < h]
259         right_peaks = [p for p in peaks if h < p < r]
260         head_vs_left = (closes[l] - closes[h]) / max(abs(closes[l]), 1e-9)
261         head_vs_right = (closes[r] - closes[h]) / max(abs(closes[r]), 1e-9)
262         if (
263             l < h < r
264             and (r - l) <= max_span
265             and _pivots_separated([l, h, r])
266             and left_peaks
267             and right_peaks
268             and closes[h] < closes[l]
269             and closes[h] < closes[r]
270             and head_vs_left >= HEAD_PROMINENCE_PCT
271             and head_vs_right >= HEAD_PROMINENCE_PCT
272             and _nearly_equal(closes[l], closes[r], tol_pct=0.025)
273         ):
274             neck = float((closes[left_peaks[-1]] + closes[right_peaks[0]]) / 2.0)
275             hits.append(
276                 {
277                     "pattern": "inverse_head_and_shoulders",
278                     "timeframe": timeframe,
279                     "indices": [l, h, r],
280                     "last_pivot": r,
281                     "prices": [float(closes[l]), float(closes[h]), float(closes[r])],
282                     "neckline": neck,
283                     "prominence": float(min(head_vs_left, head_vs_right)),
284                     "bias": "bullish",
285                 }
286             )
287
288     # Triangles on recent 40 bars: converging highs/lows
289     window = min(40, len(closes))
290     seg_high = highs[-window:]
291     seg_low = lows[-window:]
292     x = np.arange(window, dtype=float)
293     if window >= 20:
294         high_fit = np.polyfit(x, seg_high, 1)
295         low_fit = np.polyfit(x, seg_low, 1)

```

```
296     high_slope = float(high_fit[0])
297     low_slope = float(low_fit[0])
298     high_range = float(seg_high.max() - seg_high.min())
299     low_range = float(seg_low.max() - seg_low.min())
300     # Flat side: abs(OLS slope) < 15% of (side range / window bars).
301     flat_high = abs(high_slope) < (high_range / window) * 0.15
302     flat_low = abs(low_slope) < (low_range / window) * 0.15
303     rising_low = low_slope > 0
304     falling_high = high_slope < 0
305     high_touches = _count_line_touches(seg_high, high_slope, float(high_fit[1]),
306     tol_pct=TRIANGLE_TOUCH_PCT)
307     low_touches = _count_line_touches(seg_low, low_slope, float(low_fit[1]), tol_pct=TRIANGLE_TOUCH_PCT)
308     enough_touches = (
309         high_touches >= TRIANGLE_MIN_TOUCHES_PER_SIDE
310         and low_touches >= TRIANGLE_MIN_TOUCHES_PER_SIDE
311     )
312     start_idx = len(closes) - window
313     high_last = _last_touch_offset(seg_high, high_slope, float(high_fit[1]), tol_pct=TRIANGLE_TOUCH_PCT)
314     low_last = _last_touch_offset(seg_low, low_slope, float(low_fit[1]), tol_pct=TRIANGLE_TOUCH_PCT)
315     touch_offsets = [idx for idx in (high_last, low_last) if idx is not None]
316     last_pivot = start_idx + max(touch_offsets) if touch_offsets else start_idx
317     triangle_meta = {
318         "timeframe": timeframe,
319         "indices": [start_idx, last_pivot],
320         "last_pivot": last_pivot,
321         "high_slope": high_slope,
322         "low_slope": low_slope,
323         "high_touches": high_touches,
324         "low_touches": low_touches,
325         "prominence": float(min(high_range, low_range)),
326     }
327     if enough_touches and flat_high and rising_low:
328         hits.append({"pattern": "ascending_triangle", "bias": "bullish", **triangle_meta})
329     elif enough_touches and flat_low and falling_high:
330         hits.append({"pattern": "descending_triangle", "bias": "bearish", **triangle_meta})
331     elif enough_touches and falling_high and rising_low:
332         hits.append({"pattern": "symmetrical_triangle", "bias": "neutral", **triangle_meta})
333     return hits
334
335
336 def collect_pattern_hits(
337     historicals_for_symbol: dict[str, Any],
338     timeframes: list[str],
339 ) -> list[dict[str, Any]]:
340     """Daily first. 10-minute / hour only on names with a daily pattern hit."""
341     dailyBars = list(historicals_for_symbol.get("day") or historicals_for_symbol.get("daily") or [])
342     daily_hits = detect_patterns(dailyBars, timeframe="day")
343     ranked = rank_daily_setups(daily_hits)
344     if not ranked:
345         return []
346     winner = ranked[0]
347     hits = [winner]
348     for tf in timeframes:
349         if tf in ("day", "daily"):
350             continue
351         bars = historicals_for_symbol.get(tf) or []
352         hits.extend(detect_patterns(list(bars), timeframe=tf))
353     return hits
```

```
1 from __future__ import annotations
2
3 from typing import Any
4
5 from pipeline.io_util import utc_now_iso
6
7
8 def build_news_signal(
9     symbol: str,
10    articles: list[dict[str, Any]],
11    earnings: dict[str, Any] | None = None,
12 ) -> dict[str, Any]:
13     """Agent C: factual packaging of RH news/earnings payloads only."""
14     headlines = []
15     for a in articles[:10]:
16         headlines.append(
17             {
18                 "title": a.get("title") or a.get("headline"),
19                 "published_at": a.get("published_at") or a.get("updated_at") or a.get("created_at"),
20                 "source": a.get("source") or a.get("author"),
21                 "url": a.get("url") or a.get("link"),
22             }
23         )
24     return {
25         "symbol": symbol,
26         "as_of": utc_now_iso(),
27         "headline_count": len(headlines),
28         "headlines": headlines,
29         "earnings": earnings,
30         "notes": "Read-only catalyst pack; no sentiment scores invented.",
31     }
```

```

1 from __future__ import annotations
2
3 from datetime import date, datetime
4 from typing import Any
5
6 from pipeline.session import today_et
7
8
9 def _parse_ymd(value: str) -> date:
10     return datetime.strptime(value[:10], "%Y-%m-%d").date()
11
12
13 def dte(expiration: str, as_of: date | None = None) -> int:
14     as_of = as_of or today_et()
15     return (_parse_ymd(expiration) - as_of).days
16
17
18 def _instrument_type(row: dict[str, Any]) -> str:
19     raw = (row.get("type") or row.get("option_type") or "").strip().lower()
20     if raw in ("c", "call"):
21         return "call"
22     if raw in ("p", "put"):
23         return "put"
24     return raw
25
26
27 def _strike(row: dict[str, Any]) -> float:
28     value = row.get("strike_price")
29     if value in (None, ""):
30         value = row.get("strike")
31     return float(value)
32
33
34 def _instrument_id(row: dict[str, Any]) -> str | None:
35     value = row.get("id") or row.get("instrument_id")
36     return str(value) if value not in (None, "") else None
37
38
39 def instrument_id(row: dict[str, Any]) -> str | None:
40     return _instrument_id(row)
41
42
43 def strike_price(row: dict[str, Any]) -> float:
44     return _strike(row)
45
46
47 def _typed_instruments(instruments: list[dict[str, Any]], option_type: str) -> list[dict[str, Any]]:
48     want = option_type.strip().lower()
49     if want in ("c", "call"):
50         want = "call"
51     elif want in ("p", "put"):
52         want = "put"
53     return [row for row in instruments if _instrument_type(row) == want]
54
55
56 def _same_instrument(left: dict[str, Any], right: dict[str, Any]) -> bool:
57     lid, rid = _instrument_id(left), _instrument_id(right)
58     if lid and rid:
59         return lid == rid
60     return _strike(left) == _strike(right)
61
62
63 def strikes_bracket_spot(spot: float, instruments: list[dict[str, Any]], *, option_type: str) -> bool:
64     """True when the page includes strikes on both sides of spot (ATM is in the set)."""
65     typed = _typed_instruments(instruments, option_type)
66     if not typed or spot <= 0:
67         return False
68     strikes = [_strike(row) for row in typed]
69     return min(strikes) <= spot <= max(strikes)
70
71
72 def _atm_sort_key(row: dict[str, Any], spot: float, option_type: str) -> tuple[float, float]:
73     strike = _strike(row)
74     distance = abs(strike - spot)

```



```

75     ot = option_type.strip().lower()
76     # Tie: lower strike for calls, higher strike for puts.
77     if ot in ("put", "p"):
78         return (distance, -strike)
79     return (distance, strike)
80
81
82 def rank_atm_then_one_otm(
83     spot: float,
84     instruments: list[dict[str, Any]],
85     *,
86     option_type: str,
87 ) -> list[dict[str, Any]]:
88     """ATM is the nearest strike (no 1% cutoff). Then exactly one listed strike OTM from ATM."""
89     typed = _typed_instruments(instruments, option_type)
90     if not typed or spot <= 0:
91         return []
92     typed_sorted = sorted(typed, key=lambda row: _atm_sort_key(row, spot, option_type))
93     atm = typed_sorted[0]
94     ranked: list[dict[str, Any]] = [{"selection": "atm", "instrument": atm}]
95     atm_strike = _strike(atm)
96     ot = option_type.strip().lower()
97     if ot in ("call", "c"):
98         farther = sorted((row for row in typed if _strike(row) > atm_strike), key=_strike)
99     else:
100     farther = sorted((row for row in typed if _strike(row) < atm_strike), key=_strike, reverse=True)
101     if farther and not _same_instrument(farther[0], atm):
102         ranked.append({"selection": "one_otm", "instrument": farther[0]})
103     return ranked
104
105
106 def pick_atm_or_one_otm(
107     spot: float,
108     instruments: list[dict[str, Any]],
109     *,
110     option_type: str,
111 ) -> dict[str, Any] | None:
112     """Prefer ATM (nearest strike); fallback one strike OTM for calls/puts."""
113     ranked = rank_atm_then_one_otm(spot, instruments, option_type=option_type)
114     return ranked[0] if ranked else None
115
116
117 def choose_structure_from_bias(bias: str | None) -> str | None:
118     if bias == "bullish":
119         return "long_call"
120     if bias == "bearish":
121         return "long_put"
122     return None
123
124
125 def filter_expirations(
126     expiration_dates: list[str],
127     *,
128     max_dte: int,
129     min_dte: int = 2,
130     as_of: date | None = None,
131 ) -> list[str]:
132     as_of = as_of or today_et()
133     out = []
134     for exp in expiration_dates:
135         days = dte(exp, as_of=as_of)
136         if min_dte <= days <= max_dte:
137             out.append(exp)
138     return sorted(out)
139
140
141 def rank_expirations(
142     expiration_dates: list[str],
143     *,
144     overnight_holding_enabled: bool,
145     as_of: date | None = None,
146     same_day_min_dte: int = 2,
147     same_day_max_dte: int = 3,
148     overnight_min_dte: int = 4,

```

```
149     overnight_max_dte: int = 7,
150     hard_min_dte: int = 2,
151     hard_max_dte: int = 7,
152 ) -> list[str]:
153     """Deterministic expiration group. Ascending DTE inside the one permitted group."""
154     as_of = as_of or today_et()
155     if overnight_holding_enabled:
156         lo, hi = overnight_min_dte, overnight_max_dte
157     else:
158         lo, hi = same_day_min_dte, same_day_max_dte
159     lo = max(lo, hard_min_dte)
160     hi = min(hi, hard_max_dte)
161     ranked: list[tuple[int, str]] = []
162     for exp in expiration_dates:
163         days = dte(exp, as_of=as_of)
164         if lo <= days <= hi:
165             ranked.append((days, exp))
166     return [exp for _, exp in sorted(ranked)]
```

```

1 from __future__ import annotations
2
3 import re
4 from math import floor
5 from typing import Any
6
7 from pipeline.risk import equity_risk_plan
8
9 # Inverse / leveraged-short ETFs. Long index ETFs (SPY, VTI, QQQ) are allowed.
10 INVERSE_ETF_SYMBOLS = frozenset(
11     {
12         "SH",
13         "SDS",
14         "SPXU",
15         "SPXS",
16         "SPDN",
17         "PSQ",
18         "QID",
19         "SQQQ",
20         "DOG",
21         "DXD",
22         "SDOW",
23         "TZA",
24         "FAZ",
25         "SOXS",
26         "LABD",
27         "YANG",
28         "SCO",
29         "DUG",
30         "DUST",
31         "JDST",
32         "WEBS",
33         "HIBS",
34         "SARK",
35         "RWM",
36         "TWM",
37         "HDGE",
38         "EFZ",
39         "EPV",
40         "MYV",
41         "MZZ",
42         "BIS",
43         "SRTY",
44         "TYO",
45         "KOLD",
46         "SBB",
47         "SEF",
48         "SIJ",
49         "SKF",
50     }
51 )
52
53
54 # One-word UltraShort / Inverse names. Do not match "ultra short" (short-duration bond funds).
55 _INVERSE_NAME_RE = re.compile(
56     r"\b(?:inverse|ultrashort|ultrapro\s+short|leveraged\s+inverse)\b",
57     re.IGNORECASE,
58 )
59
60
61 def is_inverse_etf(symbol: str, fundamentals: dict[str, Any] | None = None) -> bool:
62     if (symbol or "").strip().upper() in INVERSE_ETF_SYMBOLS:
63         return True
64     fund = fundamentals or {}
65     blob = " ".join(
66         str(fund.get(k) or "")
67         for k in ("description", "name", "security_name", "instrument_name")
68     )
69     return bool(_INVERSE_NAME_RE.search(blob))
70
71
72 def parse_bid_ask(quote: dict[str, Any] | None) -> tuple[float | None, float | None]:
73     if not quote:
74         return None, None

```

```
75
76     def _num(*keys: str) -> float | None:
77         for key in keys:
78             if key in quote and quote[key] not in (None, ""):
79                 try:
80                     return float(quote[key])
81                 except (TypeError, ValueError):
82                     continue
83         return None
84
85     bid = _num("bid_price", "bid", "bid_last")
86     ask = _num("ask_price", "ask", "ask_last")
87     return bid, ask
88
89
90 def equity_quote_ok(
91     quote: dict[str, Any] | None,
92     *,
93     reject_one_sided: bool = True,
94 ) -> tuple[bool, str | None, dict[str, Any]]:
95     bid, ask = parse_bid_ask(quote)
96     metrics: dict[str, Any] = {"bid": bid, "ask": ask}
97     if bid is None or ask is None:
98         return False, "missing_bid_ask", metrics
99     if reject_one_sided and (bid <= 0 or ask <= 0):
100         return False, "one_sided_or_missing_quote", metrics
101     if ask < bid:
102         return False, "crossed_quote", metrics
103     mid = (bid + ask) / 2.0
104     metrics["mid"] = mid
105     metrics["spread"] = ask - bid
106     return True, None, metrics
107
108
109 def regular_hours_buy_ok(tradability: dict[str, Any] | None) -> tuple[bool, str | None]:
110     if not tradability:
111         return False, "missing_tradability"
112     if tradability.get("halted") is True:
113         return False, "halted"
114     rh = (
115         tradability.get("regular_hours")
116         or tradability.get("session_regular_hours")
117         or tradability.get("regular")
118         or {}
119     )
120     if isinstance(rh, dict) and rh:
121         buy = rh.get("buy")
122         if buy is False:
123             return False, "regular_hours_buy_false"
124         if buy is True:
125             return True, None
126         tradable = rh.get("tradable")
127         if tradable is False:
128             return False, "regular_hours_not_tradable"
129         if tradable is True:
130             return True, None
131     for key in ("tradable", "is_tradable", "can_trade"):
132         if tradability.get(key) is False:
133             return False, "not_tradable"
134         if tradability.get(key) is True:
135             return True, None
136     return False, "regular_hours_buy_not_confirmed"
137
138
139 def whole_share_size(buying_power: float, limit_price: float) -> dict[str, Any]:
140     """shares = floor(buying_power / limit). Notional must be > buying power."""
141     if buying_power is None or limit_price is None:
142         return {"ok": False, "shares": 0, "notional": 0.0, "reason": "invalid_price_or_buying_power"}
143     try:
144         bp = float(buying_power)
145         limit = float(limit_price)
146     except (TypeError, ValueError):
147         return {"ok": False, "shares": 0, "notional": 0.0, "reason": "invalid_price_or_buying_power"}
148     if bp <= 0 or limit <= 0:
```

```

149         return {"ok": False, "shares": 0, "notional": 0.0, "reason": "invalid_price_or_buying_power"}
150     shares = int(floor(bp / limit))
151     notional = shares * limit
152     if shares < 1:
153         return {"ok": False, "shares": 0, "notional": 0.0, "reason": "cannot_afford_one_share"}
154     if notional > bp + 1e-9:
155     return {"ok": False, "shares": shares, "notional": notional, "reason": "notional_exceeds_buying_power"}
156     return {"ok": True, "shares": shares, "notional": notional, "reason": None}
157
158
159 def buying_power_from_raw(raw: dict[str, Any]) -> float | None:
160     portfolio = raw.get("portfolio") if isinstance(raw.get("portfolio"), dict) else {}
161     for value in (
162         raw.get("buying_power"),
163         portfolio.get("buying_power"),
164         portfolio.get("buying_power_usd"),
165     ):
166         if value not in (None, ""):
167             try:
168                 return float(value)
169             except (TypeError, ValueError):
170                 continue
171     return None
172
173
174 def select_equity_day_trade_candidates(
175     *,
176     symbols: list[str],
177     technicals_by_symbol: dict[str, Any],
178     option_candidate_symbols: set[str],
179     quotes_by_symbol: dict[str, Any],
180     tradability_by_symbol: dict[str, Any],
181     fundamentals_by_symbol: dict[str, Any],
182     buying_power: float | None,
183     playbook_status: str,
184     take_profit_pct: float,
185     stop_loss_pct: float,
186 ) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
187     """Bullish long-share day-trade candidates. Never shorts. Options-first skip."""
188     candidates: list[dict[str, Any]] = []
189     rejected: list[dict[str, Any]] = []
190
191     for symbol in symbols:
192         bias = (technicals_by_symbol.get(symbol) or {}).get("dominant_bias")
193         fund = fundamentals_by_symbol.get(symbol) or {}
194         if is_inverse_etf(symbol, fund):
195             rejected.append({"symbol": symbol, "reason": "inverse_etf", "bias": bias})
196             continue
197         if symbol in option_candidate_symbols:
198             rejected.append({"symbol": symbol, "reason": "options_priority", "bias": bias})
199             continue
200         if bias != "bullish":
201             rejected.append({"symbol": symbol, "reason": "equity_long_only_requires_bullish", "bias": bias})
202             continue
203         ok_tr, tr_reason = regular_hours_buy_ok(tradability_by_symbol.get(symbol))
204         if not ok_tr:
205             rejected.append({"symbol": symbol, "reason": tr_reason, "bias": bias})
206             continue
207         ok_q, q_reason, q_metrics = equity_quote_ok(quotes_by_symbol.get(symbol))
208         if not ok_q:
209             rejected.append({"symbol": symbol, "reason": q_reason, "bias": bias, **q_metrics})
210             continue
211         if buying_power is None:
212             rejected.append({"symbol": symbol, "reason": "missing_buying_power", "bias": bias})
213             continue
214         limit = float(q_metrics["ask"])
215         size = whole_share_size(buying_power, limit)
216         if not size["ok"]:
217             rejected.append({"symbol": symbol, "reason": size["reason"], "bias": bias, "limit": limit,
218                             "buying_power": buying_power})
219             continue
220         plan = equity_risk_plan(
221             cost_basis=float(size["notional"]),
222             take_profit_pct=take_profit_pct,

```

```
222         stop_loss_pct=stop_loss_pct,
223         shares=size["shares"],
224         limit_price=limit,
225     )
226     candidates.append(
227         {
228             "symbol": symbol,
229             "structure": "long_shares",
230             "side": "buy",
231             "bias": bias,
232             "quantity": size["shares"],
233             "limit_price": limit,
234             "notional": size["notional"],
235             "buying_power": buying_power,
236             "bid": q_metrics["bid"],
237             "ask": q_metrics["ask"],
238             "order_type": "limit",
239             "time_in_force": "gfd",
240             "market_hours": "regular_hours",
241             "playbook_status": playbook_status,
242             "risk": plan,
243         }
244     )
245     return candidates, rejected
```

```

1 from __future__ import annotations
2
3 from typing import Any
4
5
6 def extract_greeks(quote: dict[str, Any]) -> dict[str, Any]:
7     """Copy Greeks only from Robinhood quote fields; never invent values."""
8     fields = (
9         "delta",
10        "gamma",
11        "theta",
12        "vega",
13        "rho",
14        "implied_volatility",
15    )
16    out: dict[str, Any] = {}
17    missing: list[str] = []
18    for f in fields:
19        if f in quote and quote[f] not in (None, ""):
20            try:
21                out[f] = float(quote[f])
22            except (TypeError, ValueError):
23                missing.append(f)
24        else:
25            missing.append(f)
26    return {"greeks": out, "missing_fields": missing, "source": "robinhood_get_option_quotes"}
27
28
29 def delta_in_band(
30     delta: float | None,
31     *,
32     option_type: str,
33     lo: float = 0.4,
34     hi: float = 0.5,
35 ) -> tuple[bool, str | None]:
36     """Signed long-option bands. Do not accept absolute-only or sign-inverted values."""
37     if delta is None:
38         return False, "delta_missing_from_quote"
39     ot = (option_type or "").strip().lower()
40     if ot in ("put", "p"):
41         lo_signed, hi_signed = -abs(hi), -abs(lo)
42         if lo_signed <= delta <= hi_signed:
43             return True, None
44         return False, f"put_delta_{delta:.4f}_outside_{lo_signed}_{hi_signed}"
45     if ot in ("call", "c"):
46         lo_signed, hi_signed = abs(lo), abs(hi)
47         if lo_signed <= delta <= hi_signed:
48             return True, None
49         return False, f"call_delta_{delta:.4f}_outside_{lo_signed}_{hi_signed}"
50     return False, "delta_option_type_required"

```

```

1 from __future__ import annotations
2
3 from typing import Any
4
5 OPTIONS_SL_PCT_MIN = 0.20
6 OPTIONS_SL_PCT_MAX = 0.50
7 OPTIONS_TP_PCT_MIN = 0.30
8 OPTIONS_TARGET_REWARD_TO_RISK = 2.0
9 OPTIONS_DEFAULT_SL_PCT = 0.20
10 OPTIONS_DEFAULT_TP_PCT = 0.40
11
12
13 def options_risk_plan(
14     *,
15     premium_per_share: float,
16     contracts: int = 1,
17     multiplier: float = 100.0,
18     take_profit_pct: float = OPTIONS_DEFAULT_TP_PCT,
19     stop_loss_pct: float = OPTIONS_DEFAULT_SL_PCT,
20 ) -> dict[str, Any]:
21     """Cash-risked plan for long options. Does not invent prices beyond inputs.
22
23     Locked bands: SL 20-50% of premium; TP 30-100%+ of premium; aim 1:2 R:R.
24     Owner-locked working pair is ·20% / +40% (1:2, inside the bands).
25     """
26     if contracts != 1:
27         raise ValueError("Phase rules require max 1 contract")
28     if stop_loss_pct < OPTIONS_SL_PCT_MIN or stop_loss_pct > OPTIONS_SL_PCT_MAX:
29         raise ValueError(
30 f"options stop_loss_pct must be in [{OPTIONS_SL_PCT_MIN}, {OPTIONS_SL_PCT_MAX}], got {stop_loss_pct}"
31 )
32     if take_profit_pct < OPTIONS_TP_PCT_MIN:
33         raise ValueError(
34 f"options take_profit_pct must be >= {OPTIONS_TP_PCT_MIN}, got {take_profit_pct}"
35 )
36     cash = premium_per_share * multiplier * contracts
37     reward_to_risk = take_profit_pct / stop_loss_pct if stop_loss_pct else None
38     return {
39         "asset_class": "option",
40         "contracts": contracts,
41         "premium_per_share": premium_per_share,
42         "multiplier": multiplier,
43         "cash_risked": cash,
44         "take_profit_pct": take_profit_pct,
45         "stop_loss_pct": stop_loss_pct,
46         "take_profit_value": cash * (1.0 + take_profit_pct),
47         "stop_loss_value": cash * (1.0 - stop_loss_pct),
48         "take_profit_premium": premium_per_share * (1.0 + take_profit_pct),
49         "stop_loss_premium": premium_per_share * (1.0 - stop_loss_pct),
50         "stop_loss_pct_band": {"min": OPTIONS_SL_PCT_MIN, "max": OPTIONS_SL_PCT_MAX},
51         "take_profit_pct_band": {"min": OPTIONS_TP_PCT_MIN, "uncapped": True},
52         "target_reward_to_risk": OPTIONS_TARGET_REWARD_TO_RISK,
53         "reward_to_risk": reward_to_risk,
54         "meets_target_rr": reward_to_risk is not None
55         and reward_to_risk + 1e-12 >= OPTIONS_TARGET_REWARD_TO_RISK,
56         "broker_exit": "stop_first_until_oco",
57         "monitor_take_profit_in_loop": True,
58     }
59
60
61 def equity_risk_plan(
62     *,
63     cost_basis: float,
64     take_profit_pct: float = 0.25,
65     stop_loss_pct: float = 0.2,
66     shares: int | None = None,
67     limit_price: float | None = None,
68 ) -> dict[str, Any]:
69     plan: dict[str, Any] = {
70         "asset_class": "equity",
71         "cost_basis": cost_basis,
72         "take_profit_pct": take_profit_pct,
73         "stop_loss_pct": stop_loss_pct,
74         "take_profit_value": cost_basis * (1.0 + take_profit_pct),

```



```
75     "stop_loss_value": cost_basis * (1.0 - stop_loss_pct),
76     "broker_exit": "stop_first_until_oco",
77     "monitor_take_profit_in_loop": True,
78     "flatten_before_close": True,
79     "side": "long_only",
80 }
81 if shares is not None:
82     plan["shares"] = int(shares)
83 if limit_price is not None:
84     plan["limit_price"] = limit_price
85     plan["stop_price"] = limit_price * (1.0 - stop_loss_pct)
86     plan["take_profit_price"] = limit_price * (1.0 + take_profit_pct)
87 return plan
```

```

1 from __future__ import annotations
2
3 from collections import Counter
4 from typing import Any
5
6 from pipeline.equity_day_trade import buying_power_from_raw, select_equity_day_trade_candidates
7 from pipeline.greeks import delta_in_band, extract_greeks
8 from pipeline.io_util import append_jsonl, load_rules, utc_now_iso, write_json, SIGNALS, JOURNAL
9 from pipeline.news import build_news_signal
10 from pipeline.options_structure import (
11     choose_structure_from_bias,
12     instrument_id,
13     rank_atm_then_one_otm,
14     rank_expirations,
15     strikes_bracket_spot,
16 )
17 from pipeline.patterns import collect_pattern_hits
18 from pipeline.quotes import extract_bod_nlv
19 from pipeline.risk import equity_risk_plan, options_risk_plan
20 from pipeline.session import today_et
21 from pipeline.universe import apply_liquidity_filter, extract_watchlist_symbols, option_quote_liquid
22
23 PHASE2_SNAPSHOT_NOTE = (
24     "Pipeline snapshot. Daily-pattern screen only. Not H-entry-ready: "
25     "does not confirm hour alignment, 10m breakout/retest, live trigger, "
26     "IV, volume/OI, quote age, event blackout, BOD NLV, or dual fee ceilings. "
27     "Re-quote live. Never place from this file. Agent H must ignore equity_candidates."
28 )
29
30
31 def dominant_bias(pattern_hits: list[dict[str, Any]]) -> str | None:
32     for hit in pattern_hits:
33         if hit.get("timeframe") in ("day", "daily") and hit.get("bias") in ("bullish", "bearish"):
34             return str(hit["bias"])
35     biases = [p.get("bias") for p in pattern_hits if p.get("bias") in ("bullish", "bearish")]
36     if not biases:
37         return None
38     counts = Counter(biases)
39     top, n = counts.most_common(1)[0]
40     if n > len(biases) / 2:
41         return top
42     return None
43
44
45 def _snapshot(payload: dict[str, Any]) -> dict[str, Any]:
46     return {
47         **payload,
48         "do_not_place": True,
49         "h_entry_ready": False,
50         "snapshot_note": PHASE2_SNAPSHOT_NOTE,
51     }
52
53
54 def _option_premium(quote: dict[str, Any], liq_metrics: dict[str, Any]) -> tuple[float | None, str | None]:
55     """Playbook premium is bid/ask mid. Do not require mark_price."""
56     mid = liq_metrics.get("mid")
57     if mid not in (None, ""):
58         try:
59             value = float(mid)
60             if value > 0:
61                 return value, "mid"
62         except (TypeError, ValueError):
63             pass
64     for key in ("mark_price", "adjusted_mark_price"):
65         if quote.get(key) not in (None, ""):
66             try:
67                 value = float(quote[key])
68                 if value > 0:
69                     return value, key
70             except (TypeError, ValueError):
71                 continue
72     return None, None
73

```

```

74
75 def run_pipeline(raw: dict[str, Any]) -> dict[str, Any]:
76     """
77     Phase 2 orchestrator.
78     `raw` is assembled by the Cursor agent from RH MCP responses (read-only).
79     Never places orders.
80     """
81     rules = load_rules()
82     as_of = utc_now_iso()
83     assert rules["execution"]["phase2_places_orders"] is False
84
85     # --- Agent A ---
86     universe_extract = extract_watchlist_symbols(
87         raw.get("watchlists", []),
88         raw.get("watchlist_items_by_id", {}),
89         raw.get("option_watchlist_items"),
90         include_crypto=rules["universe"]["include_crypto"],
91         include_options_watchlist=rules["universe"]["include_options_watchlist"],
92     )
93     liq = apply_liquidity_filter(
94         universe_extract["equity_symbols"],
95         raw.get("fundamentals_by_symbol", {}),
96         min_average_volume=float(rules["liquidity"]["min_average_volume"]),
97     )
98     universe_signal = {
99         "agent": "A_scanner",
100         "as_of": as_of,
101         "mode": "read_only",
102         "extract": universe_extract,
103         "liquidity": liq,
104         "index_symbols": universe_extract["index_symbols"],
105         "eligible_equities": liq["passed_symbols"],
106     }
107     write_json(SIGNALS / "universe.json", _snapshot(universe_signal))
108
109     # --- Agent B ---
110     technicals: dict[str, Any] = {"agent": "B_patterns", "as_of": as_of, "symbols": {}}
111     historicals = raw.get("historicals_by_symbol_timeframe", {})
112     for symbol in liq["passed_symbols"]:
113         symbol_hits = collect_pattern_hits(
114             historicals.get(symbol) or {},
115             list(rules["patterns"]["timeframes"]),
116         )
117         technicals["symbols"][symbol] = {
118             "pattern_hits": symbol_hits,
119             "dominant_bias": dominant_bias(symbol_hits),
120         }
121     write_json(SIGNALS / "technicals.json", _snapshot(technicals))
122
123     # --- Agent C ---
124     news_signal = {"agent": "C_news", "as_of": as_of, "symbols": {}}
125     news_raw = raw.get("news_by_symbol", {})
126     earnings_raw = raw.get("earnings_by_symbol", {})
127     for symbol in liq["passed_symbols"]:
128         news_signal["symbols"][symbol] = build_news_signal(
129             symbol,
130             news_raw.get(symbol) or [],
131             earnings_raw.get(symbol),
132         )
133     write_json(SIGNALS / "news.json", _snapshot(news_signal))
134
135     # --- Agents D + I ---
136     option_candidates: list[dict[str, Any]] = []
137     greeks_rows: list[dict[str, Any]] = []
138     equity_fallbacks: list[dict[str, Any]] = []
139     spots = raw.get("spots_by_symbol", {})
140     chains = raw.get("option_chains_by_symbol", {})
141     instruments_by_key = raw.get("option_instruments_by_symbol_exp", {})
142     quotes_by_id = raw.get("option_quotes_by_id", {})
143     quotes_by_id = {str(k): v for k, v in quotes_by_id.items()}
144     buying_power = buying_power_from_raw(raw)
145     delta_lo = float(rules["options"]["strike"]["delta_min"])
146     delta_hi = float(rules["options"]["strike"]["delta_max"])
147     max_spread = float(rules["liquidity"]["option_max_spread_pct_of_price"])

```

```

148     pref_spread = float(rules["liquidity"]["option_preferred_spread_pct_of_price"])
149     reject_one_sided = bool(rules["liquidity"]["reject_one_sided_quotes"])
150     multiplier = 100.0
151     min_dte = int(rules["options"].get("min_dte", 0))
152     max_dte = int(rules["options"]["max_dte"])
153     overnight_holding = bool(rules["agent_h"].get("overnight_holding_enabled", False))
154
155     for symbol in liq["passed_symbols"]:
156         bias = technicals["symbols"].get(symbol, {}).get("dominant_bias")
157         structure = choose_structure_from_bias(bias)
158         spot = spots.get(symbol)
159         if structure is None or spot is None:
160             equity_fallbacks.append(
161                 {
162                     "symbol": symbol,
163                     "reason": "no_directional_bias_or_spot" if structure is None else "missing_spot",
164                     "bias": bias,
165                 }
166             )
167             continue
168
169         chain = chains.get(symbol) or {}
170         expirations = rank_expirations(
171             chain.get("expiration_dates") or [],
172             overnight_holding_enabled=overnight_holding,
173             hard_min_dte=min_dte,
174             hard_max_dte=max_dte,
175         )
176         if not expirations:
177             equity_fallbacks.append({"symbol": symbol, "reason": "no_expiration_within_max_dte", "bias": bias})
178             continue
179
180         option_type = "call" if structure == "long_call" else "put"
181         passed: dict[str, Any] | None = None
182         last_reject: dict[str, Any] | None = None
183         for exp in expirations:
184             instruments = instruments_by_key.get(f"{symbol}|{exp}") or instruments_by_key.get(symbol) or []
185             if not strikes_bracket_spot(float(spot), instruments, option_type=option_type):
186                 last_reject = {
187                     "symbol": symbol,
188                     "reason": "option_chain_incomplete_atm_not_in_page",
189                     "bias": bias,
190                     "expiration": exp,
191                 }
192                 continue
193
194             ranked = rank_atm_then_one_otm(float(spot), instruments, option_type=option_type)
195             if not ranked:
196                 last_reject = {"symbol": symbol, "reason": "no_instrument_match", "bias": bias, "expiration": exp}
197                 continue
198
199             for pick in ranked:
200                 inst = pick["instrument"]
201                 oid = instrument_id(inst)
202                 if not oid:
203                     last_reject = {"symbol": symbol, "reason": "missing_option_id", "bias": bias, "expiration": exp}
204                     continue
205                 quote = quotes_by_id.get(oid) or quotes_by_id.get(str(inst.get("id"))) or {}
206                 ok_liq, liq_reason, liq_metrics = option_quote_liquid(
207                     quote,
208                     max_spread_pct_of_price=max_spread,
209                     preferred_spread_pct_of_price=pref_spread,
210                     reject_one_sided=reject_one_sided,
211                 )
212                 gpack = extract_greeks(quote)
213                 greeks_rows.append(
214                     {
215                         "symbol": symbol,
216                         "option_id": oid,
217                         "expiration": exp,
218                         "type": option_type,
219                         "strike": inst.get("strike_price", inst.get("strike")),
220                         "selection": pick["selection"],
221                         **gpack,

```

```

222         "liquidity": {"ok": ok_liq, "reason": liq_reason, **liq_metrics},
223     }
224 )
225 if not ok_liq:
226     last_reject = {
227         "symbol": symbol,
228         "reason": f"option_illiquid:{liq_reason}",
229         "bias": bias,
230         "option_id": oid,
231         "selection": pick["selection"],
232         "expiration": exp,
233     }
234     continue
235 ok_delta, delta_reason = delta_in_band(
236     gpack["greeks"].get("delta"),
237     option_type=option_type,
238     lo=delta_lo,
239     hi=delta_hi,
240 )
241 if not ok_delta:
242     last_reject = {
243         "symbol": symbol,
244         "reason": f"greeks_filter:{delta_reason}",
245         "bias": bias,
246         "option_id": oid,
247         "selection": pick["selection"],
248         "expiration": exp,
249     }
250     continue
251 premium, premium_source = _option_premium(quote, liq_metrics)
252 if premium is None:
253     last_reject = {
254         "symbol": symbol,
255         "reason": "missing_bid_ask_mid",
256         "bias": bias,
257         "option_id": oid,
258         "selection": pick["selection"],
259         "expiration": exp,
260     }
261     continue
262 cash = premium * multiplier
263 if buying_power is None:
264     last_reject = {
265         "symbol": symbol,
266         "reason": "missing_buying_power",
267         "bias": bias,
268         "option_id": oid,
269         "selection": pick["selection"],
270         "expiration": exp,
271     }
272     continue
273 if cash > buying_power + 1e-9:
274     last_reject = {
275         "symbol": symbol,
276         "reason": "exceeds_buying_power",
277         "bias": bias,
278         "option_id": oid,
279         "selection": pick["selection"],
280         "expiration": exp,
281         "cash_debit": cash,
282         "buying_power": buying_power,
283     }
284     continue
285 passed = {
286     "symbol": symbol,
287     "structure": structure,
288     "bias": bias,
289     "expiration": exp,
290     "option_type": option_type,
291     "strike": inst.get("strike_price", inst.get("strike")),
292     "option_id": oid,
293     "selection": pick["selection"],
294     "premium_mark": premium,
295     "premium_source": premium_source,

```

```

296         "greeks": gpack["greeks"],
297         "liquidity": liq_metrics,
298         "contracts": 1,
299         "cash_debit": cash,
300         "playbook_status": rules["options"]["playbook_status"],
301     }
302     break
303     if passed is not None:
304         break
305
306     if passed is None:
307 equity_fallbacks.append(last_reject or {"symbol": symbol, "reason": "no_instrument_match", "bias":
bias})
308         continue
309         option_candidates.append(passed)
310
311     write_json(
312         SIGNALS / "option_candidates.json",
313         _snapshot(
314             {
315                 "agent": "D_option_structure",
316                 "as_of": as_of,
317                 "mode": "read_only",
318                 "candidates": option_candidates,
319                 "equity_fallbacks": equity_fallbacks,
320                 "max_contracts": rules["options"]["max_contracts"],
321                 "phase2_checks": [
322                     "daily_pattern",
323                     "expiration_group",
324                     "atm_otm",
325                     "signed_delta",
326                     "spread",
327                     "buying_power",
328                 ],
329                 "h_still_requires": [
330                     "hour_confirm",
331                     "10m_breakout_retest",
332                     "live_trigger",
333                     "iv",
334                     "volume_oi",
335                     "quote_age",
336                     "event_blackout",
337                     "bod_nlv",
338                     "dual_fee_ceilings",
339                 ],
340             }
341         ),
342     )
343     write_json(
344         SIGNALS / "greeks.json",
345         _snapshot({"agent": "I_greeks", "as_of": as_of, "source": "robinhood_get_option_quotes", "rows":
greeks_rows})),
346     )
347
348     equity_candidates, equity_rejects = select_equity_day_trade_candidates(
349         symbols=liq["passed_symbols"],
350         technicals_by_symbol=technicals["symbols"],
351         option_candidate_symbols={c["symbol"] for c in option_candidates},
352         quotes_by_symbol=raw.get("equity_quotes_by_symbol") or {},
353         tradability_by_symbol=raw.get("equity_tradability_by_symbol") or {},
354         fundamentals_by_symbol=raw.get("fundamentals_by_symbol") or {},
355         buying_power=buying_power,
356         playbook_status=str(rules["risk"]["equity"]["playbook_status"]),
357         take_profit_pct=float(rules["risk"]["equity"]["take_profit_pct_of_cost"]),
358         stop_loss_pct=float(rules["risk"]["equity"]["stop_loss_pct_of_cost"]),
359     )
360     write_json(
361         SIGNALS / "equity_candidates.json",
362         _snapshot(
363             {
364                 "agent": "D_equity_day_trade",
365                 "as_of": as_of,
366                 "mode": "read_only",
367                 "playbook_status": rules["risk"]["equity"]["playbook_status"],

```

```

368         "playbook_path": rules["risk"]["equity"]["playbook_path"],
369         "side": "long_only",
370         "no_shorting": True,
371         "priority": "options_first",
372         "agent_h_may_use": False,
373         "agent_h_equity_fallback": False,
374         "candidates": equity_candidates,
375         "rejected": equity_rejects,
376         "option_fallback_notes": equity_fallbacks,
377     }
378 ),
379 )
380
381 # --- Agent E ---
382 risk_plans: list[dict[str, Any]] = []
383 for cand in option_candidates:
384     plan = options_risk_plan(
385         premium_per_share=float(cand["premium_mark"]),
386         contracts=1,
387         take_profit_pct=float(rules["risk"]["options"]["take_profit_pct_of_cash"]),
388         stop_loss_pct=float(rules["risk"]["options"]["stop_loss_pct_of_cash"]),
389     )
390     risk_plans.append({"symbol": cand["symbol"], "option_id": cand["option_id"], **plan})
391
392 # Equity fallback risk plans only when explicitly provided cost basis in raw (optional)
393 for fb in raw.get("equity_fallback_costs", []) or []:
394     plan = equity_risk_plan(
395         cost_basis=float(fb["cost_basis"]),
396         take_profit_pct=float(rules["risk"]["equity"]["take_profit_pct_of_cost"]),
397         stop_loss_pct=float(rules["risk"]["equity"]["stop_loss_pct_of_cost"]),
398     )
399     risk_plans.append({"symbol": fb["symbol"], **plan})
400
401 for cand in equity_candidates:
402     risk_plans.append({"symbol": cand["symbol"], **cand["risk"]})
403
404 write_json(
405     SIGNALS / "risk_plan.json",
406     _snapshot(
407         {
408             "agent": "E_risk",
409             "as_of": as_of,
410             "mode": "read_only",
411             "max_open_positions": rules["risk"]["max_open_positions"],
412             "plans": risk_plans,
413             "notes": "Stop-first until OCO; TP monitored in loop. Phase 2 does not place orders.",
414         }
415     ),
416 )
417
418 bod_nlv, bod_field = extract_bod_nlv(raw.get("portfolio"))
419 summary = {
420     "as_of": as_of,
421     "phase": 2,
422     "places_orders": False,
423     "h_entry_ready": False,
424     "eligible_equities": liq["passed_symbols"],
425     "option_candidate_count": len(option_candidates),
426     "equity_candidate_count": len(equity_candidates),
427     "equity_fallback_count": len(equity_fallbacks),
428     "risk_plan_count": len(risk_plans),
429     "bod_nlv": bod_nlv,
430     "bod_nlv_field": bod_field,
431     "open_questions": rules.get("open_questions", []),
432 }
433 write_json(SIGNALS / "phase2_summary.json", _snapshot(summary))
434 append_jsonl(
435     JOURNAL / "loop_runs.jsonl",
436     {"event": "phase2_cycle", "mode": "read_only", **summary},
437 )
438 journal_day = today_et().isoformat()
439 append_jsonl(
440     JOURNAL / f"{journal_day}.md.jsonl",
441     {"type": "markdown_seed", "text": f"# {journal_day} Phase 2 cycle\n\n options candidates:

```

```
    {len(option_candidates)}\n- eligible equities: {len(liq['passed_symbols'])}\n"},
442 )
443 md_path = JOURNAL / f"{journal_day}.md"
444 prev = md_path.read_text(encoding="utf-8") if md_path.exists() else f"# Trading journal
{journal_day}\n\n"
445 prev += f"\n## Phase 2 read-only cycle ({as_of})\n"
446 prev += f"- Eligible equities: {'', '.join(liq['passed_symbols']) or '(none)'}\n"
447 prev += f"- Option candidates: {len(option_candidates)}\n"
448 prev += f"- Equity day-trade candidates: {len(equity_candidates)}\n"
449 prev += f"- Equity fallbacks (option miss): {len(equity_fallbacks)}\n"
450 prev += f"- Orders placed: **none** (Phase 2 read-only)\n"
451 md_path.write_text(prev, encoding="utf-8")
452
453 return summary
```



```
1 from __future__ import annotations
2
3 from datetime import datetime
4 from typing import Any
5
6 from pipeline.io_util import JOURNAL, SIGNALS, append_jsonl, load_rules, read_json, utc_now_iso,
  write_json
7 from pipeline.session import (
8     NO_NEW_OPTION_ENTRIES_BEFORE,
9     entries_open,
10    is_rth,
11    now_et,
12    option_entries_open,
13 )
14
15
16 def load_latest_option_candidates() -> list[dict[str, Any]]:
17     path = SIGNALS / "option_candidates.json"
18     if not path.exists():
19         return []
20     payload = read_json(path)
21     if payload.get("do_not_place") or payload.get("historical"):
22         return []
23     return list(payload.get("candidates") or [])
24
25
26 def load_latest_equity_candidates() -> list[dict[str, Any]]:
27     path = SIGNALS / "equity_candidates.json"
28     if not path.exists():
29         return []
30     payload = read_json(path)
31     if payload.get("do_not_place") or payload.get("historical"):
32         return []
33     return list(payload.get("candidates") or [])
34
35
36 def can_place_live(
37     *,
38     explicit_confirm: bool,
39     playbook_released: bool,
40     playbook_kind: str = "options",
41     h_enabled: bool | None = None,
42     now: datetime | None = None,
43     h_rth_override: bool = False,
44 ) -> tuple[bool, str | None]:
45     """Agent F place-gate. Never invent a confirm. H owns RTH while enabled."""
46     if not playbook_released:
47         return False, f"{playbook_kind}_playbook_still_draft"
48     if not explicit_confirm:
49         return False, "missing_explicit_user_confirm"
50     if not is_rth(now):
51         return False, "outside_rth"
52     if playbook_kind == "options" and not option_entries_open(now):
53         if now_et(now).time() < NO_NEW_OPTION_ENTRIES_BEFORE:
54             return False, "no_new_option_entries_before_0945"
55         return False, "no_new_entries_after_1545"
56     if not entries_open(now):
57         return False, "no_new_entries_after_1545"
58     if h_enabled is None:
59         h_enabled = load_rules().get("execution", {}).get("unsupervised_agent_h") == "enabled"
60     if h_enabled and not h_rth_override:
61         return False, "h_owns_rth_while_enabled"
62     return True, None
63
64
65 def build_option_entry_proposal(
66     candidate: dict[str, Any],
67     *,
68     limit_price: str,
69     account_last4: str = "2907",
70 ) -> dict[str, Any]:
71     """Build a supervised review/place payload. Does not call the broker."""
72     return {
73         "agent": "F_supervised_execution",
```

```

74     "mode": "dry_review_until_confirm",
75     "as_of": utc_now_iso(),
76     "account_last4": account_last4,
77     "asset_class": "option",
78     "action": "buy_to_open",
79     "symbol": candidate.get("symbol"),
80     "structure": candidate.get("structure"),
81     "expiration": candidate.get("expiration"),
82     "option_type": candidate.get("option_type"),
83     "strike": candidate.get("strike"),
84     "option_id": candidate.get("option_id"),
85     "quantity": "1",
86     "order_type": "limit",
87     "price": limit_price,
88     "time_in_force": "gfd",
89     "market_hours": "regular_hours",
90     "chain_symbol": candidate.get("symbol"),
91     "underlying_type": "equity",
92     "legs": [
93         {
94             "option_id": candidate.get("option_id"),
95             "side": "buy",
96             "position_effect": "open",
97             "ratio_quantity": 1,
98         }
99     ],
100     "playbook_status": candidate.get("playbook_status"),
101     "places_order": False,
102 }
103
104
105 def build_equity_entry_proposal(
106     candidate: dict[str, Any],
107     *,
108     limit_price: str | None = None,
109     quantity: int | None = None,
110     account_last4: str = "2907",
111 ) -> dict[str, Any]:
112     """Long-only equity day-trade proposal. Never sell-to-open. Does not call the broker."""
113     shares = int(quantity if quantity is not None else candidate.get("quantity") or 0)
114     if shares < 1:
115         raise ValueError("equity day trade requires at least 1 whole share")
116     price = str(limit_price if limit_price is not None else candidate.get("limit_price"))
117     fill = float(price)
118     stop_pct = float((candidate.get("risk") or {}).get("stop_loss_pct") or 0.2)
119     tp_pct = float((candidate.get("risk") or {}).get("take_profit_pct") or 0.25)
120     return {
121         "agent": "F_supervised_execution",
122         "mode": "dry_review_until_confirm",
123         "as_of": utc_now_iso(),
124         "account_last4": account_last4,
125         "asset_class": "equity",
126         "action": "buy_to_open",
127         "side": "buy",
128         "symbol": candidate.get("symbol"),
129         "structure": "long_shares",
130         "quantity": str(shares),
131         "order_type": "limit",
132         "price": price,
133         "time_in_force": "gfd",
134         "market_hours": "regular_hours",
135         "stop_loss_pct": stop_pct,
136         "take_profit_pct": tp_pct,
137         "stop_price_after_fill": fill * (1.0 - stop_pct),
138         "take_profit_price": fill * (1.0 + tp_pct),
139         "playbook_status": candidate.get("playbook_status"),
140         "places_order": False,
141     }
142
143
144 def record_review(proposal: dict[str, Any], review_response: dict[str, Any] | None, *, error: str |
145 None = None) -> dict[str, Any]:
146     rules = load_rules()
147     asset = proposal.get("asset_class") or "option"

```

```
147     if asset == "equity":
148         released = rules["risk"]["equity"]["playbook_status"] == "RELEASED"
149         kind = "equity"
150         status = rules["risk"]["equity"]["playbook_status"]
151     else:
152         released = rules["options"]["playbook_status"] == "RELEASED"
153         kind = "options"
154         status = rules["options"]["playbook_status"]
155     allowed, reason = can_place_live(
156         explicit_confirm=False,
157         playbook_released=released,
158         playbook_kind=kind,
159         h_enabled=rules.get("execution", {}).get("unsupervised_agent_h") == "enabled",
160     )
161     record = {
162         "event": "phase3_dry_review",
163         "as_of": utc_now_iso(),
164         "proposal": proposal,
165         "review": review_response,
166         "error": error,
167         "places_order": False,
168         "playbook_status": status,
169         "place_gate": {"allowed": allowed, "reason": reason},
170     }
171     write_json(SIGNALS / "execution_review.json", record)
172     append_jsonl(JOURNAL / "reviews.jsonl", record)
173     return record
```

```
1  #!/usr/bin/env python3
2  """Run Phase 2 pipeline from a raw MCP dump JSON file (read-only)."""
3
4  from __future__ import annotations
5
6  import argparse
7  import json
8  import sys
9  from pathlib import Path
10
11  ROOT = Path(__file__).resolve().parents[1]
12  if str(ROOT) not in sys.path:
13      sys.path.insert(0, str(ROOT))
14
15  from pipeline.io_util import DATA_RAW
16  from pipeline.orchestrator import run_pipeline
17
18
19  def main() -> int:
20      parser = argparse.ArgumentParser(description="Phase 2 read-only signal pipeline")
21      parser.add_argument(
22          "--raw",
23          type=Path,
24          default=DATA_RAW / "latest_raw.json",
25          help="Path to RH MCP assembled raw JSON",
26      )
27      args = parser.parse_args()
28      if not args.raw.exists():
29          print(f"Raw file not found: {args.raw}", file=sys.stderr)
30          return 1
31      raw = json.loads(args.raw.read_text(encoding="utf-8"))
32      summary = run_pipeline(raw)
33      print(json.dumps(summary, indent=2))
34      return 0
35
36
37  if __name__ == "__main__":
38      raise SystemExit(main())
```

```
1  #!/usr/bin/env python3
2  """Build a printable HTML + concatenated Python source book of this repo.
3
4  Covers every .py module used by Agent F (this Cursor chat) and Agent H
5  (the autonomous Agentic bot). H's place-permission prompt is not Python
6  and is listed only as a pointer.
7  """
8
9  from __future__ import annotations
10
11  import html
12  from datetime import datetime, timezone
13  from pathlib import Path
14
15  ROOT = Path(__file__).resolve().parents[1]
16  DOCS = ROOT / "docs"
17
18  # (path, part, used_by, one-line role)
19  CATALOG: list[tuple[str, str, str, str]] = [
20      ("pipeline/__init__.py", "0 · Package", "F + H", "Pipeline package marker"),
21      ("pipeline/io_util.py", "1 · Shared", "F + H", "Paths, rules.json loader, journal helpers"),
22      ("pipeline/session.py", "1 · Shared", "F + H", "RTH clock; option entries 09:45:15:45; H lease is on
23      origin/main"),
24      ("pipeline/orders.py", "1 · Shared", "F + H", "Working-order states for Robinhood MCP (no open=true)"),
25      ("pipeline/quotes.py", "1 · Shared", "F + H", "5s underlying executable price; BOD NLV field extract"),
26      ("pipeline/fees.py", "1 · Shared", "F + H", "Dual fee ceilings: 0.49% planned loss and 0.50% with
27      fees"),
28      ("pipeline/universe.py", "2 · Agent A", "F + H", "Watchlist extract, crypto drop, inverse-ETF reject,
29      ADV · 2,000,000"),
30      ("pipeline/patterns.py", "3 · Agent B", "F + H", "Daily-first H&S / double-triple / triangle; no
31      1m/3m/5m"),
32      ("pipeline/news.py", "4 · Agent C", "F + H", "Factual RH news/earnings pack; no invented sentiment"),
33      ("pipeline/options_structure.py", "5 · Agent D", "F + H", "Long call/put; ATM/OTM; 2·3 DTE while
34      overnight off"),
35      ("pipeline/equity_day_trade.py", "5 · Agent D", "F only", "Long shares only; inverse-ETF denylist; H
36      has no equity fallback"),
37      ("pipeline/greeks.py", "6 · Agent I", "F + H", "Copy RH Greeks only; signed call +0.40·+0.50 / put
38      ·0.50·-0.40"),
39      ("pipeline/risk.py", "7 · Agent E", "F + H", "Options ·20%/+40%; equity ·20%/+25%; stop first until
40      OCO"),
41      ("pipeline/orchestrator.py", "8 · Agent G", "F + H", "Phase 2 read-only snapshot; h_entry_ready is
42      always false"),
43      ("pipeline/execution.py", "9 · Agent F", "F (chat)", "Supervised place-gate: confirm, RTH, 09:45
44      options, H-owns-RTH"),
45      ("scripts/run_phase2_cycle.py", "10 · CLI", "F + H", "Load data/raw/latest_raw.json and run the
46      orchestrator"),
47      ("scripts/build_python_source_book.py", "10 · CLI", "docs", "This generator · rebuilds the printable
48      source book"),
49      ("pipeline/tests/test_phase2.py", "11 · Tests", "CI / F", "Universe, liquidity, signed delta, ATM/OTM,
50      expiration rank"),
51      ("pipeline/tests/test_orders.py", "11 · Tests", "CI / F", "Working states and locked agent_h schema"),
52      ("pipeline/tests/test_fees.py", "11 · Tests", "CI / F", "Dual NLV fee ceilings"),
53      ("pipeline/tests/test_session.py", "11 · Tests", "CI / F", "ET calendar date and flatten window"),
54      ("pipeline/tests/test_equity_day_trade.py", "11 · Tests", "CI / F", "Long-only equity selection and
55      Phase 2 snapshots"),
56      ("pipeline/tests/test_execution.py", "11 · Tests", "CI / F", "F place-gate including 09:45 option
57      lock"),
58  ]
59
60  def _read(rel: str) -> str:
61      path = ROOT / rel
62      if not path.exists():
63          return ""
64      return path.read_text(encoding="utf-8")
65
66  def _banner(rel: str, part: str, used_by: str, role: str) -> str:
67      line = "=" * 72
68      return (
69          f"# {line}\n"
70          f"# {rel}\n"
71          f"# Part: {part}\n"
72          f"# Used by: {used_by}\n"
73      )
```

```

60         f"# {role}\n"
61         f"# {line}\n\n"
62     )
63
64
65 def _catalog() -> list[tuple[str, str, str, str]]:
66     return [row for row in CATALOG if (ROOT / row[0]).exists() and _read(row[0]).strip()]
67
68
69 def build_python_book() -> str:
70     now = datetime.now(timezone.utc).replace(microsecond=0).isoformat()
71     chunks: list[str] = [
72         "# ruff: noqa\n",
73         "# pylint: skip-file\n",
74         """PRINTABLE SOURCE BOOK · do not import or execute this file.\n",
75         "\n",
76         "Agentic trading program: Agent F (supervised Cursor chat) and\n",
77         "Agent H (autonomous Agentic bot) share this Python pipeline.\n",
78         "Live place_* is Robinhood MCP, not a side effect of this code.\n",
79         "H's standing prompt is playbooks/agent_h_autonomous.PROMPT.md (not Python).\n",
80         "\n",
81         f"Generated: {now}\n",
82         "Print companion: docs/agentic-python-source-printable.html\n",
83         """\n\n",
84     ]
85     for rel, part, used_by, role in _catalog():
86         chunks.append(_banner(rel, part, used_by, role))
87         chunks.append(_read(rel).rstrip() + "\n\n")
88     return "".join(chunks)
89
90
91 def build_html(py_book: str) -> str:
92     now = datetime.now(timezone.utc).replace(microsecond=0).strftime("%Y-%m-%d %H:%M UTC")
93     toc_rows = []
94     sections = []
95     total_lines = 0
96     for rel, part, used_by, role in _catalog():
97         src = _read(rel)
98         n = src.count("\n") + (0 if src.endswith("\n") or not src else 1)
99         total_lines += n
100         anchor = rel.replace("/", "-").replace(".", "-")
101         toc_rows.append(
102             f"<tr><td>{html.escape(part)}</td><td><a href='#{anchor}'>"
103             f"<code>{html.escape(rel)}</code></a></td>"
104             f"<td>{html.escape(used_by)}</td><td>{n}</td>"
105             f"<td>{html.escape(role)}</td></tr>"
106         )
107         numbered = []
108         lines = src.splitlines()
109         width = max(3, len(str(len(lines))))
110         for i, line in enumerate(lines, 1):
111             numbered.append(
112                 f"<span class='ln'>{i:{width}d}</span> {html.escape(line)}"
113             )
114         sections.append(
115             f"<section class='file' id='{anchor}'>"
116             f"<h2>{html.escape(rel)}</h2>"
117             f"<p class='meta'><strong>{html.escape(part)}</strong> · {html.escape(used_by)}"
118             f" · {n} lines · {html.escape(role)}</p>"
119             f"<pre class='source'>{chr(10).join(numbered)}\n</pre>"
120             f"</section>"
121         )
122
123     return f"""<!DOCTYPE html>
124 <html lang="en">
125 <head>
126 <meta charset="utf-8" />
127 <meta name="viewport" content="width=device-width, initial-scale=1" />
128 <title>Agentic Python Source Book (Printable)</title>
129 <style>
130 :root {{ --ink:#111; --muted:#333; --line:#222; --bg:#fff; --box:#f4f4f4; }}
131 * {{ box-sizing: border-box; }}
132 body {{
133     margin: 0; color: var(--ink); background: var(--bg);

```

```

134     font-family: "IBM Plex Sans", "Segoe UI", Helvetica, Arial, sans-serif;
135     font-size: 10.4pt; line-height: 1.35;
136 }
137 .toolbar {{
138     position: sticky; top: 0; z-index: 20; display: flex; gap: 12px;
139     align-items: center; background: #111; color: #fff;
140     padding: 10px 16px; font-size: 13px;
141 }}
142 .toolbar button {{
143     background: #fff; color: #111; border: 0; padding: 8px 14px;
144     font-weight: 700; cursor: pointer;
145 }}
146 .page {{ max-width: 8.5in; margin: 0 auto; padding: 0.5in 0.55in 0.65in; }}
147 h1 {{ font-size: 20pt; margin: 0 0 0.08in; letter-spacing: -0.02em; }}
148 h2 {{
149     font-size: 12pt; margin: 0.28in 0 0.08in; padding-bottom: 0.04in;
150     border-bottom: 1.5pt solid var(--line); page-break-after: avoid;
151 }}
152 h3 {{ font-size: 11pt; margin: 0.16in 0 0.06in; page-break-after: avoid; }}
153 .kicker {{ font-size: 9.5pt; letter-spacing: 0.08em; text-transform: uppercase; }}
154 .rule {{ width: 1.3in; height: 3px; background: #111; margin: 0.12in 0 0.18in; }}
155 .meta, .note, .footer {{ font-size: 9.3pt; color: var(--muted); }}
156 .meta strong {{ color: var(--ink); }}
157 .badge-row {{ display: flex; flex-wrap: wrap; gap: 6px; margin: 0.1in 0 0.16in; }}
158 .badge {{
159     border: 1pt solid var(--line); padding: 3px 8px; font-size: 8.4pt; background: var(--box);
160 }}
161 table {{ width: 100%; border-collapse: collapse; font-size: 8.8pt; margin: 0 0 0.14in; }}
162 th, td {{ border: 1pt solid var(--line); padding: 0.045in 0.06in; vertical-align: top; text-align:
left; }}
163 th {{ background: #e8e8e8; font-weight: 700; }}
164 .card {{
165     border: 1pt solid var(--line); padding: 0.1in 0.12in; background: var(--box);
166     page-break-inside: avoid; margin-bottom: 0.1in;
167 }}
168 ul {{ margin: 0.04in 0 0.08in; padding-left: 0.2in; }}
169 li {{ margin: 0.03in 0; }}
170 code {{ font-family: "IBM Plex Mono", Consolas, "Courier New", monospace; font-size: 8.8pt; }}
171 pre.source {{
172     font-family: "IBM Plex Mono", Consolas, "Courier New", monospace;
173     font-size: 7.15pt; line-height: 1.28; white-space: pre-wrap; word-break: break-word;
174     background: var(--box); border: 1pt solid var(--line); padding: 0.09in 0.1in;
175     margin: 0 0 0.08in;
176 }}
177 pre.source .ln {{ color: #888; margin-right: 0.12in; user-select: none; }}
178 .file {{ page-break-before: always; }}
179 .footer {{ margin-top: 0.18in; padding-top: 0.08in; border-top: 1pt solid #999; font-size: 8.2pt; }}
180 @media print {{
181     .no-print {{ display: none !important; }}
182     .page {{ max-width: none; padding: 0; }}
183     a {{ color: inherit; text-decoration: none; }}
184     h2, table, .card {{ break-inside: avoid; }}
185     pre.source {{ font-size: 7pt; }}
186 }}
187 @page {{ size: Letter portrait; margin: 0.45in; }}
188 </style>
189 </head>
190 <body>
191     <div class="toolbar no-print">
192         <button type="button" onclick="window.print()">Print / Save as PDF</button>
193         <span>Agentic Python source · Letter portrait · {total_lines} lines · {len(CATALOG)} files</span>
194     </div>
195     <div class="page">
196         <p class="kicker">Jarrod Besner · Agentic ....2907</p>
197         <h1>Python source book</h1>
198         <div class="rule"></div>
199         <p class="meta">Chat agent <strong>F</strong> and autonomous bot <strong>H</strong> share this
pipeline.
200         Generated {html.escape(now)}. Companion file: <code>docs/agentik_python_source_book.py</code>.</p>
201         <div class="badge-row">
202             <span class="badge">{len(CATALOG)} Python files</span>
203             <span class="badge">{total_lines} lines</span>
204             <span class="badge">Language: Python 3</span>
205             <span class="badge">Does not place orders</span>

```

```

206     </div>
207
208     <h3>Who uses which code</h3>
209     <div class="card">
210         <ul>
211             <li><strong>Agent F (this Cursor chat)</strong> · supervised. Reads the pipeline, reviews tickets via
212             <code>pipeline/execution.py</code>, and only calls Robinhood <code>place_*</code> after an explicit
213             confirm of a specific order. Blocked during RTH while H is enabled.</li>
214             <li><strong>Agent H (Agentic AI Bot)</strong> · unsupervised Cursor Automation. The standing prompt is
215             <code>playbooks/agent_h_autonomous.PROMPT.md</code> (markdown, not Python). On each fire H checks out
216             <code>main</code>, reads <code>config/rules.json</code> · <code>agent_h</code>, and uses
217             daily · 1-hour · completed 10-minute · live quote only (no 1m / 3m / 5m).
218             Live <code>place_*</code> is Robinhood MCP from that prompt, not a pipeline side effect.
219             H is options-only; it must ignore equity candidates.</li>
220             <li><strong>Not in this book:</strong> lock JSON, playbooks, and the H prompt. Those are not
                Python.</li>
221         </ul>
222     </div>
223
224     <h3>Contents</h3>
225     <table>
226         <thead><tr><th>Part</th><th>File</th><th>Used by</th><th>Lines</th><th>Role</th></tr></thead>
227         <tbody>
228             { ''.join(toc_rows) }
229         </tbody>
230     </table>
231     <p class="note">Each file starts on a new printed page. Source is verbatim Python from the repo.</p>
232     { ''.join(sections) }
233     <p class="footer">Agentic Python source book · {html.escape(now)} · {total_lines} lines in
        {len(CATALOG)} files ·
234     print from this HTML or open <code>docs/agentic_python_source_book.py</code>.</p>
235     </div>
236 </body>
237 </html>
238 """
239
240
241 def build_pdf(path: Path) -> None:
242     import pymupdf
243
244     page_w, page_h = 612, 792
245     margin = 36
246     body_font = 7.2
247     header_font = 9.5
248     cover_title = 22
249     line_h = 9.4
250     usable_h = page_h - margin * 2 - 18
251     max_lines = int(usable_h / line_h)
252     max_chars = 108
253
254     doc = pymupdf.open()
255     now = datetime.now(timezone.utc).replace(microsecond=0).strftime("%Y-%m-%d %H:%M UTC")
256
257     def new_page():
258         return doc.new_page(width=page_w, height=page_h)
259
260     def footer(page, label: str) -> None:
261         page.insert_text(
262             pymupdf.Point(margin, page_h - 18),
263             f"Agentic Python source · {label} · {now} · page {page.number + 1}",
264             fontname="helv",
265             fontsize=7,
266             color=(0.25, 0.25, 0.25),
267         )
268
269     def wrap(text: str, width: int) -> list[str]:
270         if len(text) <= width:
271             return [text]
272         words = text.replace("\t", " ").split(" ")
273         lines: list[str] = []
274         cur = ""
275         for word in words:
276             trial = word if not cur else f"{cur} {word}"
277             if len(trial) <= width:

```

```

278         cur = trial
279         continue
280     if cur:
281         lines.append(cur)
282     while len(word) > width:
283         lines.append(word[:width])
284         word = word[width:]
285     cur = word
286 if cur:
287     lines.append(cur)
288 return lines or [""]
289
290 # Cover
291 page = new_page()
292 y = 72
293 page.insert_text(pymupdf.Point(margin, y), "JARROD BESNER · AGENTIC", fontname="helv", fontsize=10)
294 y += 36
295 page.insert_text(pymupdf.Point(margin, y), "Python source book", fontname="helv", fontsize=cover_title)
296 y += 28
297 page.draw_rect(pymupdf.Rect(margin, y, margin + 90, y + 3), fill=(0, 0, 0), width=0)
298 y += 28
299 cover_lines = [
300     "All Python used by Agent F (supervised Cursor chat) and Agent H",
301     "(autonomous Agentic bot). Live place_* is Robinhood MCP, not a",
302     "side effect of this code. H's standing prompt is markdown:",
303     "playbooks/agent_h_autonomous.PROMPT.md · not included here.",
304     "",
305     f"{len(CATALOG)} files · generated {now}",
306     "Language: Python 3 · Letter portrait",
307 ]
308 for line in cover_lines:
309     page.insert_text(pymupdf.Point(margin, y), line, fontname="helv", fontsize=11)
310     y += 16
311 y += 10
312 page.insert_text(pymupdf.Point(margin, y), "Contents", fontname="helv", fontsize=13)
313 y += 18
314 for rel, part, used_by, role in _catalog():
315     n = _read(rel).count("\n") + 1
316     row = f"{part:16s} {rel:42s} {used_by:10s} {n:4d} {role}"
317     for w in wrap(row, 98):
318         if y > page_h - 48:
319             footer(page, "cover")
320             page = new_page()
321             y = margin + 12
322         page.insert_text(pymupdf.Point(margin, y), w, fontname="cour", fontsize=7)
323         y += 10
324 footer(page, "cover")
325
326 for rel, part, used_by, role in _catalog():
327     src_lines = _read(rel).splitlines()
328     page = new_page()
329     header = f"{rel} · {part} · {used_by} · {role}"
330 page.insert_text(pymupdf.Point(margin, margin + 4), header[:110], fontname="helv",
331                 fontsize=header_font)
332 y = margin + 18
333 page.draw_line(pymupdf.Point(margin, y), pymupdf.Point(page_w - margin, y), color=(0, 0, 0), width=0.8)
334 y += 12
335 used = 0
336 width = max(3, len(str(len(src_lines))))
337 for i, raw in enumerate(src_lines, 1):
338     prefix = f"{i:{width}d} "
339     wrapped = wrap(raw.replace("\t", "    "), max_chars - len(prefix)) or [""]
340     for j, chunk in enumerate(wrapped):
341         if used >= max_lines:
342             footer(page, rel)
343             page = new_page()
344             page.insert_text(
345                 pymupdf.Point(margin, margin + 4),
346                 f"{rel} (continued)",
347                 fontname="helv",
348                 fontsize=header_font,
349             )
350             y = margin + 18
351         page.draw_line(

```

```
351             pymupdf.Point(margin, y),
352             pymupdf.Point(page_w - margin, y),
353             color=(0, 0, 0),
354             width=0.8,
355         )
356         y += 12
357         used = 0
358         line = prefix + chunk if j == 0 else (" " * len(prefix)) + chunk
359         page.insert_text(pymupdf.Point(margin, y), line, fontname="cour", fontsize=body_font)
360         y += line_h
361         used += 1
362         footer(page, rel)
363
364     path.parent.mkdir(parents=True, exist_ok=True)
365     doc.save(path)
366     doc.close()
367
368
369 def main() -> int:
370     DOCS.mkdir(parents=True, exist_ok=True)
371     py_book = build_python_book()
372     html_doc = build_html(py_book)
373     py_path = DOCS / "agentic_python_source_book.py"
374     html_path = DOCS / "agentic-python-source-printable.html"
375     py_path.write_text(py_book, encoding="utf-8")
376     html_path.write_text(html_doc, encoding="utf-8")
377     print(f"wrote {py_path} ({py_path.stat().st_size} bytes)")
378     print(f"wrote {html_path} ({html_path.stat().st_size} bytes)")
379     pdf_candidates = [Path("/dev/shm/agentic-python-source.pdf"), DOCS / "agentic-python-source.pdf"]
380     pdf_path = pdf_candidates[0]
381     try:
382         build_pdf(pdf_path)
383         print(f"wrote {pdf_path} ({pdf_path.stat().st_size} bytes)")
384         dest = DOCS / "agentic-python-source.pdf"
385         if pdf_path != dest:
386             dest.write_bytes(pdf_path.read_bytes())
387             print(f"copied {dest} ({dest.stat().st_size} bytes)")
388     except OSError as exc:
389         print(f"pdf write skipped: {exc}")
390     return 0
391
392
393 if __name__ == "__main__":
394     raise SystemExit(main())
```

```

1 from datetime import date, datetime, timezone
2 from zoneinfo import ZoneInfo
3
4 from pipeline.greeks import delta_in_band, extract_greeks
5 from pipeline.options_structure import (
6     filter_expirations,
7     pick_atm_or_one_otm,
8     rank_atm_then_one_otm,
9     rank_expirations,
10    strikes_bracket_spot,
11 )
12 from pipeline.patterns import rank_daily_setups
13 from pipeline.quotes import executable_underlying_price, extract_bod_nlv
14 from pipeline.patterns import collect_pattern_hits, detect_patterns
15 from pipeline.risk import equity_risk_plan, options_risk_plan
16 from pipeline.session import today_et
17 from pipeline.universe import apply_liquidity_filter, extract_watchlist_symbols, option_quote_liquid
18
19
20 def test_extract_excludes_crypto():
21     watchlists = [{"id": "1", "display_name": "My First List"}]
22     items = {
23         "1": [
24             {"object_type": "instrument", "symbol": "AAPL"},
25             {"object_type": "currency_pair", "symbol": "BTC-USD"},
26         ]
27     }
28     out = extract_watchlist_symbols(watchlists, items, [], include_crypto=False)
29     assert out["equity_symbols"] == ["AAPL"]
30     assert "BTC-USD" in out["skipped_crypto"]
31
32
33 def test_liquidity_volume_gate():
34     fund = {"AAA": {"average_volume": 3_000_000}, "BBB": {"average_volume": 1000}}
35     out = apply_liquidity_filter(["AAA", "BBB"], fund, min_average_volume=2_000_000)
36     assert out["passed_symbols"] == ["AAA"]
37     assert out["rejected"][0]["symbol"] == "BBB"
38
39
40 def test_liquidity_rejects_inverse_etfs_before_volume():
41     fund = {
42         "AAA": {"average_volume": 3_000_000},
43         "SQQQ": {"average_volume": 20_000_000, "name": "ProShares UltraPro Short QQQ"},
44         "XYZ": {"average_volume": 5_000_000, "description": "Leveraged inverse daily"},
45     }
46     out = apply_liquidity_filter(["AAA", "SQQQ", "XYZ"], fund, min_average_volume=2_000_000)
47     assert out["passed_symbols"] == ["AAA"]
48     reasons = {row["symbol"]: row["reason"] for row in out["rejected"]}
49     assert reasons["SQQQ"] == "inverse_etf"
50     assert reasons["XYZ"] == "inverse_etf"
51
52
53 def test_option_spread_gate():
54     preferred, reason, pref_m = option_quote_liquid(
55         {"bid_price": "1.00", "ask_price": "1.05"},
56         max_spread_pct_of_price=0.1,
57         preferred_spread_pct_of_price=0.05,
58     )
59     assert preferred and reason is None
60     assert pref_m["spread_quality"] == "preferred"
61
62     acceptable, reason_ok, acc_m = option_quote_liquid(
63         {"bid": "1.00", "ask": "1.08"},
64         max_spread_pct_of_price=0.1,
65         preferred_spread_pct_of_price=0.05,
66     )
67     assert acceptable and reason_ok is None
68     assert acc_m["spread_quality"] == "acceptable"
69
70     missing, missing_reason, _ = option_quote_liquid(
71         {},
72         max_spread_pct_of_price=0.1,
73         preferred_spread_pct_of_price=0.05,
74     )

```

```
75     assert not missing and missing_reason == "missing_bid_ask"
76
77     bad, reason2, _ = option_quote_liquid(
78         {"bid_price": "1.00", "ask_price": "1.50"},
79         max_spread_pct_of_price=0.1,
80         preferred_spread_pct_of_price=0.05,
81     )
82     assert not bad and reason2 == "spread_too_wide"
83
84     # $0.10 absolute on a cheap contract is ~22% of mid · reject (no dollar override).
85     cheap, cheap_reason, cheap_m = option_quote_liquid(
86         {"bid_price": "0.40", "ask_price": "0.50"},
87         max_spread_pct_of_price=0.1,
88         preferred_spread_pct_of_price=0.05,
89     )
90     assert not cheap and cheap_reason == "spread_too_wide"
91     assert cheap_m["spread_pct_of_price"] > 0.1
92
93
94 def test_double_bottom_detection():
95     # Equal troughs separated by a bounce; pad so extrema order=3 works.
96     prices = (
97         [6, 6, 6, 5, 4, 3, 4, 5, 6, 5, 4, 3, 4, 5, 6]
98         + [6.2 + i * 0.05 for i in range(20)]
99     )
100     bars = [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
101     hits = detect_patterns(bars, timeframe="day")
102     names = {h["pattern"] for h in hits}
103     assert "double_bottom" in names
104
105
106 def test_greeks_no_invention():
107     q = {"delta": "0.45", "gamma": "0.01"}
108     pack = extract_greeks(q)
109     assert pack["greeks"]["delta"] == 0.45
110     assert "theta" in pack["missing_fields"]
111     ok, _ = delta_in_band(0.45, option_type="call", lo=0.4, hi=0.5)
112     assert ok
113     bad, reason = delta_in_band(0.9, option_type="call", lo=0.4, hi=0.5)
114     assert not bad and reason is not None
115     put_ok, _ = delta_in_band(-0.45, option_type="put", lo=0.4, hi=0.5)
116     assert put_ok
117     inverted, inv_reason = delta_in_band(0.45, option_type="put", lo=0.4, hi=0.5)
118     assert not inverted and inv_reason is not None
119     abs_call, abs_reason = delta_in_band(-0.45, option_type="call", lo=0.4, hi=0.5)
120     assert not abs_call and abs_reason is not None
121
122
123 def test_options_risk_math():
124     plan = options_risk_plan(premium_per_share=2.0, contracts=1)
125     assert plan["cash_risked"] == 200.0
126     assert plan["take_profit_pct"] == 0.40
127     assert plan["stop_loss_pct"] == 0.20
128     assert plan["take_profit_value"] == 280.0
129     assert plan["stop_loss_value"] == 160.0
130     assert plan["reward_to_risk"] == 2.0
131     assert plan["meets_target_rr"] is True
132
133
134 def test_options_risk_bands():
135     wide = options_risk_plan(premium_per_share=1.0, stop_loss_pct=0.50, take_profit_pct=1.00)
136     assert wide["stop_loss_value"] == 50.0
137     assert wide["take_profit_value"] == 200.0
138     assert wide["reward_to_risk"] == 2.0
139     try:
140         options_risk_plan(premium_per_share=1.0, stop_loss_pct=0.07, take_profit_pct=0.50)
141         raise AssertionError("expected ValueError for SL below 20%")
142     except ValueError as exc:
143         assert "stop_loss_pct" in str(exc)
144     try:
145         options_risk_plan(premium_per_share=1.0, stop_loss_pct=0.25, take_profit_pct=0.20)
146         raise AssertionError("expected ValueError for TP below 30%")
147     except ValueError as exc:
148         assert "take_profit_pct" in str(exc)
```

```
149
150
151 def test_equity_risk_math():
152     plan = equity_risk_plan(cost_basis=1000.0)
153     assert plan["take_profit_value"] == 1250.0
154     assert plan["stop_loss_value"] == 800.0
155
156
157 def test_atm_pick_and_dte():
158     instruments = [
159         {"id": "1", "type": "call", "strike_price": "100"},
160         {"id": "2", "type": "call", "strike_price": "105"},
161         {"id": "3", "type": "put", "strike_price": "95"},
162     ]
163     pick = pick_atm_or_one_otm(100.0, instruments, option_type="call")
164     assert pick["selection"] == "atm"
165     assert pick["instrument"]["id"] == "1"
166     exps = filter_expirations(["2099-01-01", "2026-09-02"], max_dte=7, as_of=date(2026, 8, 30))
167     assert exps == ["2026-09-02"]
168     default_min = filter_expirations(
169         ["2026-08-30", "2026-08-31", "2026-09-01", "2026-09-02"],
170         max_dte=7,
171         as_of=date(2026, 8, 30),
172     )
173     assert default_min == ["2026-09-01", "2026-09-02"]
174     locked = filter_expirations(
175         ["2026-08-30", "2026-08-31", "2026-09-02"],
176         min_dte=2,
177         max_dte=7,
178         as_of=date(2026, 8, 30),
179     )
180     assert locked == ["2026-09-02"]
181
182
183 def test_atm_is_nearest_strike_even_when_more_than_one_percent_away():
184     instruments = [
185         {"id": "itm", "type": "call", "strike_price": "5.0"},
186         {"id": "otm", "type": "call", "strike_price": "5.5"},
187     ]
188     ranked = rank_atm_then_one_otm(5.10, instruments, option_type="call")
189     assert ranked[0]["selection"] == "atm"
190     assert ranked[0]["instrument"]["id"] == "itm"
191     assert ranked[1]["selection"] == "one_otm"
192     assert ranked[1]["instrument"]["id"] == "otm"
193     pick = pick_atm_or_one_otm(5.10, instruments, option_type="call")
194     assert pick["instrument"]["id"] == "itm"
195
196
197 def test_put_atm_then_distinct_one_otm():
198     instruments = [
199         {"id": "otm", "option_type": "put", "strike": "18.0"},
200         {"id": "atm", "option_type": "put", "strike": "18.5"},
201     ]
202     ranked = rank_atm_then_one_otm(18.30, instruments, option_type="put")
203     assert ranked[0]["instrument"]["id"] == "atm"
204     assert ranked[1]["instrument"]["id"] == "otm"
205
206
207 def test_strikes_must_bracket_spot_for_atm_page():
208     only_low = [{"id": "1", "type": "call", "strike_price": "100"}]
209     assert not strikes_bracket_spot(200.0, only_low, option_type="call")
210     assert strikes_bracket_spot(100.0, only_low, option_type="call")
211     bracketed = only_low + [{"id": "2", "type": "call", "strike_price": "210"}]
212     assert strikes_bracket_spot(200.0, bracketed, option_type="call")
213
214
215 def test_today_et_uses_new_york_calendar_not_utc():
216     utc_after_midnight = datetime(2026, 9, 1, 2, 0, tzinfo=timezone.utc)
217     assert today_et(utc_after_midnight).isoformat() == "2026-08-31"
218     et_evening = datetime(2026, 8, 31, 22, 0, tzinfo=ZoneInfo("America/New_York"))
219     assert today_et(et_evening).isoformat() == "2026-08-31"
220
221
222 def test_atm_tie_and_otm_are_measured_from_atm():
```

```
223     instruments = [
224         {"id": "low", "type": "call", "strike_price": "100"},
225         {"id": "high", "type": "call", "strike_price": "101"},
226         {"id": "higher", "type": "call", "strike_price": "102"},
227     ]
228     call_ranked = rank_atm_then_one_otm(100.5, instruments, option_type="call")
229     assert call_ranked[0]["instrument"]["id"] == "low"
230     assert call_ranked[1]["instrument"]["id"] == "high"
231
232     puts = [
233         {"id": "low", "option_type": "put", "strike": "100"},
234         {"id": "high", "option_type": "put", "strike": "101"},
235         {"id": "higher", "option_type": "put", "strike": "102"},
236     ]
237     put_ranked = rank_atm_then_one_otm(100.5, puts, option_type="put")
238     assert put_ranked[0]["instrument"]["id"] == "high"
239     assert put_ranked[1]["instrument"]["id"] == "low"
240
241     call_above_spot = rank_atm_then_one_otm(100.6, instruments, option_type="call")
242     assert call_above_spot[0]["instrument"]["id"] == "high"
243     assert call_above_spot[1]["instrument"]["id"] == "higher"
244
245
246     def test_expiration_rank_uses_same_day_group_when_overnight_off():
247         dates = ["2026-09-07", "2026-09-08", "2026-09-09", "2026-09-11"]
248         as_of = date(2026, 9, 5)
249         same_day = rank_expirations(dates, overnight_holding_enabled=False, as_of=as_of)
250         assert same_day == ["2026-09-07", "2026-09-08"]
251         overnight = rank_expirations(dates, overnight_holding_enabled=True, as_of=as_of)
252         assert overnight == ["2026-09-09", "2026-09-11"]
253
254
255     def test_underlying_quote_and_bod_nlv_helpers():
256         now = datetime(2026, 9, 4, 17, 30, tzinfo=timezone.utc)
257         fresh = {
258             "bid_price": "10.00",
259             "ask_price": "10.10",
260             "last_trade_price": "10.04",
261             "updated_at": "2026-09-04T17:29:58Z",
262         }
263         price, reason = executable_underlying_price(fresh, now=now)
264         assert reason is None
265         assert price == 10.04
266         outside = dict(fresh, last_trade_price="10.50")
267         mid, mid_reason = executable_underlying_price(outside, now=now)
268         assert mid_reason is None
269         assert mid == 10.05
270         stale, stale_reason = executable_underlying_price(
271             dict(fresh, updated_at="2026-09-04T17:29:50Z"), now=now
272         )
273         assert stale is None and stale_reason == "underlying_quote_stale"
274         amount, field = extract_bod_nlv({"start_of_day_equity": "1500.00", "total_value": "1512"})
275         assert amount == 1500.0 and field == "start_of_day_equity"
276         missing, missing_field = extract_bod_nlv({"total_value": "1512"})
277         assert missing is None and missing_field is None
278
279
280     def test_daily_setup_rank_is_deterministic():
281         hits = [
282             {
283                 "pattern": "ascending_triangle",
284                 "timeframe": "day",
285                 "bias": "bullish",
286                 "indices": [20, 80],
287                 "last_pivot": 80,
288                 "prominence": 9.0,
289             },
290             {
291                 "pattern": "head_and_shoulders",
292                 "timeframe": "day",
293                 "bias": "bearish",
294                 "indices": [10, 20, 30],
295                 "last_pivot": 30,
296                 "prominence": 0.02,
```

```
297         },
298         {
299             "pattern": "double_bottom",
300             "timeframe": "day",
301             "bias": "bullish",
302             "indices": [10, 40],
303             "last_pivot": 40,
304             "prominence": 1.0,
305         },
306         {
307             "pattern": "symmetrical_triangle",
308             "timeframe": "day",
309             "bias": "neutral",
310             "indices": [30, 50],
311             "prominence": 9.0,
312         },
313     ]
314     ranked = rank_daily_setups(hits)
315     assert [row["pattern"] for row in ranked] == [
316         "head_and_shoulders",
317         "double_bottom",
318         "ascending_triangle",
319     ]
320
321
322 def test_double_rejects_span_beyond_max_duration():
323     prices = [5.0] * 3 + [3.0] + [5.0] * 70 + [3.0] + [5.0] * 6
324     bars = [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
325     hits = detect_patterns(bars, timeframe="day")
326     assert "double_bottom" not in {h["pattern"] for h in hits}
327
328
329 def test_intraday_patterns_ignored_without_daily_hit():
330     prices = [6, 6, 6, 5, 4, 3, 4, 5, 6, 5, 4, 3, 4, 5, 6] + [6.2 + i * 0.05 for i in range(20)]
331     bars = [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
332     hits = collect_pattern_hits({"10minute": bars}, ["10minute", "hour", "day"])
333     assert hits == []
334     daily_first = collect_pattern_hits({"day": bars, "10minute": bars}, ["10minute", "hour", "day"])
335     assert any(h["timeframe"] == "day" for h in daily_first)
336     assert any(h["timeframe"] == "10minute" for h in daily_first)
```

```
1 from pipeline.equity_day_trade import INVERSE_ETF_SYMBOLS
2 from pipeline.orders import (
3     EQUITY_WORKING_STATES,
4     OPTION_WORKING_STATES,
5     has_working_orders,
6     working_orders,
7 )
8 from pipeline.io_util import load_rules
9
10
11 def test_working_option_and_equity_states():
12     rows = working_orders(
13         option_orders=[
14             {"id": "o1", "state": "queued"},
15             {"id": "o2", "state": "filled"},
16             {"id": "o3", "status": "pending_cancelled"},
17         ],
18         equity_orders=[
19             {"id": "e1", "state": "new"},
20             {"id": "e2", "state": "cancelled"},
21             {"id": "e3", "state": "unconfirmed"},
22         ],
23     )
24     ids = {row["id"] for row in rows}
25     assert ids == {"o1", "o3", "e1", "e3"}
26     assert has_working_orders([{"state": "confirmed"}], [])
27     assert not has_working_orders([{"state": "rejected"}], [{"state": "filled"}])
28
29
30 def test_rules_json_matches_working_states_and_10minute():
31     rules = load_rules()
32     assert set(rules["orders"]["option_working_states"]) == set(OPTION_WORKING_STATES)
33     assert set(rules["orders"]["equity_working_states"]) == set(EQUITY_WORKING_STATES)
34     assert rules["patterns"]["timeframes"] == ["10minute", "hour", "day"]
35     assert rules["historicals"]["intraday_interval"] == "10minute"
36     assert "15minute" not in rules["patterns"]["timeframes"]
37     assert rules["options"]["may_hold_overnight_with_stop"] is False
38     assert rules["options"]["flatten_at_close"] is True
39     assert rules["options"]["overnight_lock_confirmed"] == "2026-09-05"
40     assert rules["loop"]["flatten_equity_before_close"] is True
41     assert rules["loop"]["options_may_hold_overnight_with_stop"] is False
42     assert "flatten_before_close" not in rules["loop"]
43     assert rules["git"]["work_on"] == "main"
44     assert rules["git"]["open_pull_request"] is False
45     assert rules["git"]["create_feature_branch"] is False
46     assert rules["agent_h"]["equity_fallback"] is False
47     assert rules["agent_h"]["min_dte"] == 2
48     assert rules["agent_h"]["max_dte"] == 7
49     assert rules["agent_h"]["allow_0dte"] is False
50     assert rules["agent_h"]["allow_1dte"] is False
51     assert rules["agent_h"]["no_new_entries_before"] == "09:45"
52     assert rules["agent_h"]["same_day_expiry_target_flatten_by"] == "15:30"
53     assert rules["agent_h"]["same_day_expiry_absolute_deadline"] == "15:45"
54     assert rules["agent_h"]["max_new_entries_per_day"] == 2
55     assert rules["agent_h"]["max_planned_loss_pct_of_current_nlv"] == 0.005
56     assert rules["agent_h"]["max_debit_pct_of_current_nlv"] == 0.025
57     assert rules["agent_h"]["max_daily_realized_loss_pct_of_session_start_nlv"] == 0.01
58     assert rules["agent_h"]["option_quote"]["min_open_interest"] == 500
59     assert rules["agent_h"]["option_quote"]["max_quote_age_seconds"] == 5
60     assert rules["agent_h"]["intraday_timeframes"] == ["10minute", "hour", "day"]
61     assert rules["agent_h"]["forbidden_timeframes"] == [
62         "1minute",
63         "3minute",
64         "5minute",
65         "15minute",
66     ]
67     assert rules["agent_h"]["chart_hierarchy"] == [
68         "daily_setup",
69         "hour_confirmation",
70         "completed_10minute_trigger",
71         "live_quote",
72         "option_review",
73     ]
74     assert rules["agent_h"]["chart_roles"]["day"] == "major_trend_support_resistance_and_chart_pattern"
```



```

75 assert rules["agent_h"]["chart_roles"]["hour"] ==
    "confirm_direction_reject_conflict_with_broader_intraday_trend"
76 assert rules["agent_h"]["chart_roles"]["10minute"] ==
    "confirm_breakout_volume_retest_and_entry_trigger"
77 assert rules["agent_h"]["chart_roles"]["live_quote"] ==
    "validate_underlying_trigger_and_price_option_immediately_before_order"
78     assert rules["agent_h"]["no_1m_3m_autonomous_noise"] is True
79     assert rules["agent_h"]["no_5m_stateless_inconsistency"] is True
80     assert rules["agent_h"]["include_index_options"] is False
81     assert rules["agent_h"]["schema_version"] == "2026-09-05.6"
82     assert rules["agent_h"]["overnight"]["evaluate"] == "current_dte_each_run"
83     assert rules["agent_h"]["overnight"]["current_dte_lte_3_flatten_by"] == "15:45"
84     assert rules["agent_h"]["overnight"]["current_dte_gte_4_overnight_with_stop"] is False
85     assert rules["agent_h"]["overnight_holding_enabled"] is False
86     assert rules["agent_h"]["overnight"]["overnight_requires_verified_gtc_stop"] is True
87     assert rules["agent_h"]["overnight"]["expiration_day_absolute_deadline"] == "15:45"
88     assert rules["agent_h"]["overnight"]["dte_1_to_3_liquidation_begin"] == "15:40"
89     assert rules["agent_h"]["lease_valid_only_after_successful_push_to_origin_main"] is True
90     assert rules["agent_h"]["recheck_remote_lease_before_every_place_option_order"] is False
91     assert rules["agent_h"]["recheck_remote_lease_before_every_new_entry_place_option_order"] is True
92     assert rules["agent_h"]["failed_lease_push_means_place_nothing_and_exit"] is False
93     assert rules["agent_h"]["failed_lease_acquire_push_means_place_nothing_and_exit"] is True
94     assert rules["agent_h"]["rejected_lease_push_means_place_nothing_and_exit"] is False
95     assert rules["agent_h"]["rejected_lease_acquire_push_means_place_nothing_and_exit"] is True
96     assert rules["agent_h"]["no_review_or_place_until_remote_lease_verified"] is False
97     assert rules["agent_h"]["no_new_entry_review_or_place_until_remote_lease_verified"] is True
98     assert rules["agent_h"]["failed_lease_renewal_blocks_new_entry_only"] is True
99     assert rules["agent_h"]["failed_lease_renewal_must_still_protect_or_flatten_this_run_fill"] is True
100 assert rules["agent_h"]["protection_or_flatten_of_this_run_fill_allowed_after_failed_renewal"] is True
101 assert
    rules["agent_h"]["this_run_fill_must_protect_or_flatten_even_if_remote_lease_mismatched_or_expired"] is
    True
102     assert rules["agent_h"]["after_pull_or_rebase_reread_remote_lease_before_writing_h_lease"] is True
103 assert rules["agent_h"]["acquire_retry_uses_remote_lease_only_not_this_run_working_tree_write"] is True
104     assert rules["agent_h"]["this_run_working_tree_lease_write_does_not_block_acquire_retry"] is True
105     assert rules["agent_h"]["never_overwrite_unexpired_remote_lease_held_by_other_run_id"] is True
106     assert rules["agent_h"]["fast_forward_pull_of_other_run_lease_is_held_not_free"] is True
107 assert rules["agent_h"]["pull_ff_only_or_rebase_onto_origin_main_before_every_main_journal_push"] is
    True
108     assert rules["agent_h"]["non_fast_forward_lease_or_journal_push_retry_once_after_rebase"] is True
109     assert rules["agent_h"]["never_force_push_on_non_fast_forward"] is True
110     assert rules["agent_h"]["scheduler_must_enforce_max_concurrent_runs"] is False
111     assert rules["agent_h"]["git_lease_is_the_concurrency_gate"] is True
112     assert rules["agent_h"]["lease_acquire_before_account_or_scan"] is True
113     assert rules["agent_h"]["lease_ttl_minutes"] == 12
114     assert rules["agent_h"]["lease_renew_if_fewer_than_minutes_remaining"] == 3
115     assert rules["agent_h"]["never_force_push_or_overwrite_conflicting_lease"] is True
116     assert rules["agent_h"]["failed_lease_acquire_must_not_modify_or_clear_lease"] is True
117     assert rules["agent_h"]["only_matching_run_id_may_renew_or_release_lease"] is True
118     assert rules["agent_h"]["remote_lease_must_contain_exact_fields"] == [
119         "automation_id",
120         "run_id",
121         "started_et",
122         "expires_et",
123     ]
124     assert rules["agent_h"]["apply_both_fee_ceilings_on_every_trade"] is True
125     assert rules["agent_h"]["bod_nlv_unavailable_means_no_new_entry"] is True
126     assert rules["priority_does_not_authorize_agent_h_equity"] is True
127     assert rules["agent_h"]["session_start_required_for_new_entry"] == [
128         "et_trading_date",
129         "first_valid_rth_timestamp_et",
130         "account",
131         "bod_nlv",
132         "bod_nlv_field",
133         "daily_loss_limit_usd",
134     ]
135     assert rules["agent_h"]["session_start_diagnostic_only_fields"] == ["first_fire_baseline_nlv"]
136     assert (
137         rules["agent_h"]["forced_liquidation"]["leftover_4_to_7_dte_while_overnight_disabled"]
138         == "treat_as_dte_1_to_3_begin_1540"
139     )
140     assert rules["agent_h"]["protective_stop"]["time_in_force"] == "gtc"
141     assert rules["agent_h"]["cancel_lifecycle"]["never_assume_cancel_means_zero_fill"] is True
142     assert rules["agent_h"]["entry_order"]["cancel_confirm_before_replacement"] is True

```

```

143     assert rules["agent_h"]["take_profit"]["cancel_existing_stop_and_confirm_before_tp"] is True
144     assert rules["agent_h"]["take_profit"]["cancel_unfilled_replacement_seconds_after_initial_tp"] == 30
145     assert rules["agent_h"]["forced_liquidation"]["dte_1_to_3_begin"] == "15:40"
146     assert rules["agent_h"]["patterns"]["daily_neckline_governs_10m_breakout"] is True
147     assert rules["agent_h"]["patterns"]["overlapping_rank"][0] == "hs_then_double_triple_then_triangle"
148     assert rules["agent_h"]["patterns"]["earliest_practical_entry_after_retest_et"] == "13:10"
149     assert rules["agent_h"]["expiration_selection"]["same_day_group_dte"] == [2, 3]
150     assert rules["options"]["strike"]["put_delta_min"] == -0.5
151     assert rules["options"]["strike"]["call_otm"] == "exactly_one_listed_strike_above_atm"
152     assert "get_realized_pnl" in rules["agent_h"]["required_tools"]
153     assert "get_earnings_calendar" in rules["agent_h"]["required_tools"]
154     assert rules["agent_h"]["patterns"]["breakout_volume_multiple_of_median"] == 1.5
155     assert rules["agent_h"]["patterns"]["retest_tolerance_pct"] == 0.002
156     assert rules["agent_h"]["patterns"]["live_trigger_beyond_breakout_pct"] == 0.001
157     assert rules["agent_h"]["patterns"]["volume_statistic"] == "median"
158     assert rules["agent_h"]["estimated_round_trip_fees"] == "3 * entry_fee_from_review"
159     assert rules["agent_h"]["entry_fee_source_hierarchy"][0] == "valid_positive_total_fee_only"
160     assert rules["agent_h"]["entry_fee_source_hierarchy"][1] ==
161         "zero_total_plus_positive_component_is_fee_conflict"
162     assert rules["agent_h"]["entry_fee_source_hierarchy"][2] ==
163         "zero_total_and_zero_or_absent_components_is_explicit_zero"
164     assert rules["agent_h"]["journal_fee_conflict"] is True
165     assert rules["agent_h"]["journal_entry_fee_source"] is True
166     assert rules["agent_h"]["fee_status_values"] == [
167         "quoted",
168         "explicit_zero",
169         "ambiguous",
170         "unavailable",
171     ]
172     assert rules["agent_h"]["zero_total_plus_positive_component"]["fee_status"] == "ambiguous"
173     assert rules["agent_h"]["zero_total_plus_positive_component"]["journal"] == "fee_conflict"
174     assert rules["agent_h"]["zero_total_plus_positive_component"]["do_not_sum_or_select_estimate"] is True
175     assert rules["agent_h"]["do_not_apply_049_ceiling_when_positive_fee_included"] is False
176     assert rules["agent_h"]["universal_fee_gate"] == [
177         "planned_loss <= 0.49% current NLV",
178         "planned_loss + estimated_round_trip_fees <= 0.50% current NLV",
179     ]
180     assert rules["agent_h"]["closed_trade_uses_actual_net_pnl_not_estimated_fees"] is True
181     assert rules["agent_h"]["allow_0dte"] is False
182     assert rules["universe"]["include_index_options"] is False
183     assert rules["options"]["min_dte"] == 2
184
185 def test_permissions_is_kill_switch_not_a_rules_copy():
186     import json
187     from pathlib import Path
188
189     perms = json.loads((Path(__file__).resolve().parents[2] / "config" /
190         "autonomous_permissions.json").read_text())
191     assert perms["status"] == "ACTIVE"
192     assert perms["rules_path"] == "config/rules.json"
193     assert "universe" not in perms
194     assert "risk" not in perms
195     assert "patterns" not in perms
196
197 def test_agent_h_prompt_locks_schema_and_live_safety():
198     from pathlib import Path
199
200     prompt = (Path(__file__).resolve().parents[2] / "playbooks" /
201         "agent_h_autonomous.PROMPT.md").read_text()
202     assert "2026-09-05.6" in prompt
203     assert "2026-09-05.5" not in prompt
204     assert "2026-09-05.4" not in prompt
205     assert "2026-09-05.3" not in prompt
206     assert "The lease is not acquired unless its commit successfully pushes to" in prompt
207     assert "journal/h_lease.json` from `origin/main`, not merely the local checkout." in prompt
208     assert "Never force-push or overwrite a conflicting lease." in prompt
209     assert "A run that failed to acquire the lease must not clear or modify the lease." in prompt
210     assert "Only the run whose `run_id` matches the remote lease may renew or release it." in prompt
211     assert "renew the lease before it has fewer than" in prompt
212     assert "git pull --ff-only origin main" in prompt
213     assert "rebase that commit onto `origin/main`" in prompt
214     assert "Failed renewal is not a kill-switch ban on those recovery tickets." in prompt

```

```
213 assert "A fast-forward pull that brought in another run's lease is a **held** lease" in prompt or "if
    **another** unexpired `run_id` is there, that is a **held** lease" in prompt
214     assert "***does not** block the retry" in prompt
215     assert "re-read **only** `origin/main:journal/h_lease.json`" in prompt
216     assert "without modifying `journal/h_lease.json`" in prompt
217 assert "including when the" in prompt and "lease expired, is unreadable, another `run_id` now holds it"
    in prompt
218     assert "you **must still** place protection or flatten for that fill" in prompt
219     assert "## Cursor/Grok concurrency rule" in prompt
220     assert "***A. Clock.** Now in `America/New_York`. Clock only. **No RH calls.**" in prompt
221     assert "Git on `origin/main` is the required concurrency" in prompt
222     assert "acquire and remotely verify lease" in prompt
223     assert "time_in_force=gtc" in prompt
224     assert "09:30:09:44:59" in prompt
225     assert "approximately **13:10:15:45 ET**" in prompt
226     assert "Call delta: **+0.40 through +0.50 inclusive**" in prompt
227     assert "Put delta: **.0.50 through .0.40 inclusive**" in prompt
228     assert "planned_loss` . **0.49% of current NLV**" in prompt
229     assert "bod_nlv_unavailable" in prompt
230     assert "maximum concurrent runs = 1" not in prompt
231
232
233 def test_inverse_etf_denylist_has_no_duplicate_twm():
234     from pathlib import Path
235
236     src = Path(__file__).resolve().parents[1] / "equity_day_trade.py"
237     assert src.read_text(encoding="utf-8").count("TWM") == 1
238     assert "TWM" in INVERSE ETF_SYMBOLS
```

```
1 from pipeline.fees import classify_review_fees, fee_aware_planned_loss_ok
2
3
4 NLV = 1500.0
5 CEILING_049 = 0.0049 * NLV # 7.35
6 CEILING_050 = 0.005 * NLV # 7.50
7
8
9 def test_positive_total_fee_is_used_alone():
10     out = classify_review_fees({"total_fee": "0.65", "commission": "0.65", "sec_fee": "0.02"})
11     assert out["fee_status"] == "quoted"
12     assert out["entry_fee"] == 0.65
13     assert out["journal"] == "total_fee"
14     assert out["estimated_round_trip_fees"] == out["entry_fee"] * 3
15     assert out["apply_049_ceiling"] is True
16
17
18 def test_zero_total_plus_positive_component_is_fee_conflict():
19     out = classify_review_fees({"total_fee": "$0.00", "commission": "0.65"})
20     assert out["fee_status"] == "ambiguous"
21     assert out["entry_fee"] is None
22     assert out["journal"] == "fee_conflict"
23     assert out["estimated_exit_fee"] is None
24     assert out["estimated_round_trip_fees"] is None
25     assert out["apply_049_ceiling"] is True
26     assert fee_aware_planned_loss_ok(
27         planned_loss=CEILING_049, current_nlv=NLV, classification=out
28     )
29     assert not fee_aware_planned_loss_ok(
30         planned_loss=7.40, current_nlv=NLV, classification=out
31     )
32
33
34 def test_zero_total_and_zero_or_absent_components_is_explicit_zero():
35     accepted = classify_review_fees({"total_fee": 0, "commission": "0.00"})
36     assert accepted["fee_status"] == "explicit_zero"
37     assert accepted["entry_fee"] == 0.0
38     assert accepted["journal"] == "fee_explicit_zero"
39     assert accepted["apply_049_ceiling"] is True
40
41     absent = classify_review_fees({"total_fee": "0.00"})
42     assert absent["fee_status"] == "explicit_zero"
43     assert absent["entry_fee"] == 0.0
44     assert fee_aware_planned_loss_ok(
45         planned_loss=CEILING_049, current_nlv=NLV, classification=absent
46     )
47     assert not fee_aware_planned_loss_ok(
48         planned_loss=7.40, current_nlv=NLV, classification=absent
49     )
50
51
52 def test_missing_total_sums_non_overlapping_components():
53     out = classify_review_fees({"commission": "0.65", "sec_fee": "0.02", "taf_fee": "0.01"})
54     assert out["fee_status"] == "quoted"
55     assert out["entry_fee"] == 0.68
56     assert out["journal"].startswith("components:")
57     assert out["apply_049_ceiling"] is True
58
59
60 def test_both_fee_ceilings_apply_on_every_trade():
61     out = classify_review_fees({"total_fee": 0.03})
62     # 7.40 exceeds 0.49% of $1,500 even though 7.40 + 3*0.03 still fits 0.50%.
63     assert out["fee_status"] == "quoted"
64     assert out["apply_049_ceiling"] is True
65     assert 7.40 > CEILING_049
66     assert 7.40 + 0.09 <= CEILING_050
67     assert not fee_aware_planned_loss_ok(
68         planned_loss=7.40, current_nlv=NLV, classification=out
69     )
70     assert fee_aware_planned_loss_ok(
71         planned_loss=CEILING_049, current_nlv=NLV, classification=out
72     )
73     expensive = classify_review_fees({"total_fee": 1.00})
74     assert not fee_aware_planned_loss_ok(
```

```
75         planned_loss=CEILING_049, current_nlv=NLV, classification=expensive
76     )
77     assert fee_aware_planned_loss_ok(
78         planned_loss=4.50, current_nlv=NLV, classification=expensive
79     )
80
81
82 def test_nested_zero_total_with_positive_component_is_conflict():
83     out = classify_review_fees({"fees": {"total_fee": 0, "contract_fee": 0.12}})
84     assert out["journal"] == "fee_conflict"
85     assert out["entry_fee"] is None
86
87
88 def test_subtotal_plus_parts_is_unavailable():
89     out = classify_review_fees({"estimated_fee": "0.70", "commission": "0.65"})
90     assert out["fee_status"] == "unavailable"
91     assert out["journal"] == "fee_unavailable"
92     assert out["apply_049_ceiling"] is True
```

```
1 from datetime import datetime, timezone
2 from zoneinfo import ZoneInfo
3
4 from pipeline.session import flatten_window, today_et
5
6
7 def test_today_et_before_utc_date_rollover():
8     utc = datetime(2026, 9, 5, 0, 42, tzinfo=timezone.utc)
9     assert today_et(utc).isoformat() == "2026-09-04"
10
11
12 def test_flatten_window_is_rth_only():
13     et = ZoneInfo("America/New_York")
14     assert flatten_window(datetime(2026, 8, 31, 15, 50, tzinfo=et))
15     assert not flatten_window(datetime(2026, 8, 31, 16, 0, tzinfo=et))
```

```
1 from datetime import datetime
2 from zoneinfo import ZoneInfo
3
4 import pytest
5
6 from pipeline.equity_day_trade import (
7     equity_quote_ok,
8     is_inverse_etf,
9     regular_hours_buy_ok,
10    select_equity_day_trade_candidates,
11    whole_share_size,
12 )
13 from pipeline.execution import build_equity_entry_proposal, can_place_live
14 from pipeline.risk import equity_risk_plan
15 from pipeline.session import (
16     ET,
17     entries_open,
18     flatten_window,
19     is_rth,
20     option_entries_open,
21     session_gate,
22 )
23
24
25 def test_session_rth_and_1545_cutoff():
26     sunday_noon = datetime(2026, 8, 30, 12, 0, tzinfo=ET)
27     monday_open = datetime(2026, 8, 31, 9, 30, tzinfo=ET)
28     monday_morning = datetime(2026, 8, 31, 10, 0, tzinfo=ET)
29     monday_1545 = datetime(2026, 8, 31, 15, 45, tzinfo=ET)
30     monday_close = datetime(2026, 8, 31, 16, 0, tzinfo=ET)
31     monday_pre = datetime(2026, 8, 31, 9, 29, tzinfo=ET)
32
33     assert not is_rth(sunday_noon)
34     assert is_rth(monday_open)
35     assert is_rth(monday_morning)
36     assert entries_open(monday_open)
37     assert not option_entries_open(monday_open)
38     assert entries_open(monday_morning)
39     assert option_entries_open(monday_morning)
40     assert is_rth(monday_1545)
41     assert not entries_open(monday_1545)
42     assert not option_entries_open(monday_1545)
43     assert flatten_window(monday_1545)
44     assert not is_rth(monday_close)
45     assert not is_rth(monday_pre)
46     gate = session_gate(monday_1545)
47     assert gate["reason"] == "no_new_entries_after_1545"
48     assert gate["option_reason"] == "no_new_entries_after_1545"
49     open_gate = session_gate(monday_open)
50     assert open_gate["entries_open"] is True
51     assert open_gate["option_entries_open"] is False
52     assert open_gate["option_reason"] == "no_new_option_entries_before_0945"
53
54
55 def test_whole_share_size_uses_floor_and_buying_power_cap():
56     sized = whole_share_size(1500.0, 347.5)
57     assert sized["ok"] is True
58     assert sized["shares"] == 4
59     assert sized["notional"] == 1390.0
60     unaffordable = whole_share_size(1500.0, 1975.0)
61     assert unaffordable["ok"] is False
62     assert unaffordable["reason"] == "cannot_afford_one_share"
63     assert whole_share_size(0, 10)["ok"] is False
64
65
66 def test_no_shorting_skips_bearish_and_inverse_etfs():
67     assert is_inverse_etf("SQQQ") is True
68     assert is_inverse_etf("SPY") is False
69     assert is_inverse_etf("XYZ", {"description": "ProShares UltraShort Inverse"}) is True
70     assert is_inverse_etf("QID", {"name": "ProShares UltraShort QQQ"}) is True
71     assert is_inverse_etf("BIL", {"description": "SPDR Bloomberg 1-3 Month T-Bill Ultra Short Duration"})
72         is False
73
74     cands, rejected = select_equity_day_trade_candidates(
```

```

74     symbols=["SQQQ", "AMD", "AAPL"],
75     technicals_by_symbol={
76         "SQQQ": {"dominant_bias": "bullish"},
77         "AMD": {"dominant_bias": "bearish"},
78         "AAPL": {"dominant_bias": "bullish"},
79     },
80     option_candidate_symbols=set(),
81     quotes_by_symbol={
82         "AAPL": {"bid_price": "100", "ask_price": "100.10"},
83         "AMD": {"bid_price": "50", "ask_price": "50.05"},
84         "SQQQ": {"bid_price": "10", "ask_price": "10.02"},
85     },
86     tradability_by_symbol={
87         "AAPL": {"regular_hours": {"buy": True}},
88         "AMD": {"regular_hours": {"buy": True}},
89         "SQQQ": {"regular_hours": {"buy": True}},
90     },
91     fundamentals_by_symbol={},
92     buying_power=1500.0,
93     playbook_status="RELEASED",
94     take_profit_pct=0.25,
95     stop_loss_pct=0.2,
96 )
97 reasons = {row["symbol"]: row["reason"] for row in rejected}
98 assert reasons["SQQQ"] == "inverse_etf"
99 assert reasons["AMD"] == "equity_long_only_requires_bullish"
100 assert [c["symbol"] for c in cand] == ["AAPL"]
101 assert cand[0]["side"] == "buy"
102 assert cand[0]["quantity"] == 14 # floor(1500 / 100.10)
103
104
105 def test_options_priority_and_quote_gates():
106     cand, rejected = select_equity_day_trade_candidates(
107         symbols=["GOOGL", "META"],
108         technicals_by_symbol={
109             "GOOGL": {"dominant_bias": "bullish"},
110             "META": {"dominant_bias": "bullish"},
111         },
112         option_candidate_symbols={"GOOGL"},
113         quotes_by_symbol={"META": {"bid_price": "0", "ask_price": "350"}},
114         tradability_by_symbol={
115             "GOOGL": {"regular_hours": {"buy": True}},
116             "META": {"regular_hours": {"buy": True}},
117         },
118         fundamentals_by_symbol={},
119         buying_power=1500.0,
120         playbook_status="RELEASED",
121         take_profit_pct=0.25,
122         stop_loss_pct=0.2,
123     )
124     reasons = {row["symbol"]: row["reason"] for row in rejected}
125     assert reasons["GOOGL"] == "options_priority"
126     assert reasons["META"] == "one_sided_or_missing_quote"
127     assert cand == []
128
129
130 def test_equity_quote_and_tradability_parsers():
131     ok, reason, metrics = equity_quote_ok({"bid_price": "10", "ask_price": "10.05"})
132     assert ok and reason is None and metrics["ask"] == 10.05
133     bad, bad_reason, _ = equity_quote_ok({"bid_price": "10", "ask_price": "0"})
134     assert not bad and bad_reason == "one_sided_or_missing_quote"
135     assert regular_hours_buy_ok({"regular_hours": {"buy": True}}) == (True, None)
136     assert regular_hours_buy_ok(None)[0] is False
137     assert regular_hours_buy_ok({"halted": True})[0] is False
138
139
140 def test_equity_risk_prices_from_limit():
141     plan = equity_risk_plan(cost_basis=1000.0, shares=10, limit_price=100.0)
142     assert plan["take_profit_value"] == 1250.0
143     assert plan["stop_loss_value"] == 800.0
144     assert plan["stop_price"] == 80.0
145     assert plan["take_profit_price"] == 125.0
146     assert plan["side"] == "long_only"
147

```



```
148
149 def test_equity_place_gate_and_proposal_is_buy_only():
150     sunday = datetime(2026, 8, 30, 12, 0, tzinfo=ZoneInfo("America/New_York"))
151     monday = datetime(2026, 8, 31, 10, 0, tzinfo=ZoneInfo("America/New_York"))
152     ok, reason = can_place_live(
153         explicit_confirm=True,
154         playbook_released=False,
155         playbook_kind="equity",
156         h_enabled=False,
157         now=sunday,
158     )
159     assert not ok and reason == "equity_playbook_still_draft"
160     ok2, reason2 = can_place_live(
161         explicit_confirm=False,
162         playbook_released=True,
163         playbook_kind="equity",
164         h_enabled=False,
165         now=sunday,
166     )
167     assert not ok2 and reason2 == "missing_explicit_user_confirm"
168     sunday_blocked, sunday_reason = can_place_live(
169         explicit_confirm=True,
170         playbook_released=True,
171         playbook_kind="equity",
172         h_enabled=False,
173         now=sunday,
174     )
175     assert not sunday_blocked and sunday_reason == "outside_rth"
176     ok3, reason3 = can_place_live(
177         explicit_confirm=True,
178         playbook_released=True,
179         playbook_kind="equity",
180         h_enabled=False,
181         now=monday,
182     )
183     assert ok3 and reason3 is None
184     blocked, blocked_reason = can_place_live(
185         explicit_confirm=True,
186         playbook_released=True,
187         playbook_kind="equity",
188         h_enabled=True,
189         now=monday,
190     )
191     assert not blocked and blocked_reason == "h_owns_rth_while_enabled"
192
193     proposal = build_equity_entry_proposal(
194         {
195             "symbol": "AAPL",
196             "quantity": 4,
197             "limit_price": 100.0,
198             "playbook_status": "RELEASED",
199             "risk": {"stop_loss_pct": 0.2, "take_profit_pct": 0.25},
200         }
201     )
202     assert proposal["side"] == "buy"
203     assert proposal["action"] == "buy_to_open"
204     assert proposal["quantity"] == "4"
205     assert proposal["places_order"] is False
206     assert proposal["market_hours"] == "regular_hours"
207     assert proposal["stop_price_after_fill"] == 80.0
208     assert proposal["take_profit_price"] == 125.0
209
210
211 def test_pipeline_writes_equity_candidate_without_placing(tmp_path, monkeypatch):
212     import json
213
214     import pipeline.orchestrator as orch
215
216     signals = tmp_path / "signals"
217     journal = tmp_path / "journal"
218     signals.mkdir()
219     journal.mkdir()
220     monkeypatch.setattr(orch, "SIGNALS", signals)
221     monkeypatch.setattr(orch, "JOURNAL", journal)
```

```

222
223 prices = [6, 6, 6, 5, 4, 3, 4, 5, 6, 5, 4, 3, 4, 5, 6] + [6.2 + i * 0.05 for i in range(20)]
224 bars = [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
225 raw = {
226     "watchlists": [{"id": "1", "display_name": "T"}],
227     "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "AAPL"}]},
228     "fundamentals_by_symbol": {"AAPL": {"average_volume": 3_000_000}},
229     "historicals_by_symbol_timeframe": {"AAPL": {"day": bars}},
230     "buying_power": 1500.0,
231     "equity_quotes_by_symbol": {"AAPL": {"bid_price": "100.00", "ask_price": "100.10"}},
232     "equity_tradability_by_symbol": {"AAPL": {"regular_hours": {"buy": True}}},
233 }
234 summary = orch.run_pipeline(raw)
235 assert summary["places_orders"] is False
236 assert summary["option_candidate_count"] == 0
237 assert summary["equity_candidate_count"] == 1
238 payload = json.loads((signals / "equity_candidates.json").read_text())
239 assert payload["do_not_place"] is True
240 assert payload["h_entry_ready"] is False
241 assert payload["agent_h_may_use"] is False
242 assert summary["h_entry_ready"] is False
243 cand = payload["candidates"][0]
244 assert cand["symbol"] == "AAPL"
245 assert cand["side"] == "buy"
246 assert cand["structure"] == "long_shares"
247 assert cand["quantity"] == 14
248 assert cand["playbook_status"] == "RELEASED"
249
250
251 def _double_bottom_bars():
252     prices = [6, 6, 6, 5, 4, 3, 4, 5, 6, 5, 4, 3, 4, 5, 6] + [6.2 + i * 0.05 for i in range(20)]
253     return [{"close": c, "high": c + 0.2, "low": c - 0.2} for c in prices]
254
255
256 def _pipeline_dirs(tmp_path, monkeypatch):
257     import pipeline.orchestrator as orch
258
259     signals = tmp_path / "signals"
260     journal = tmp_path / "journal"
261     signals.mkdir()
262     journal.mkdir()
263     monkeypatch.setattr(orch, "SIGNALS", signals)
264     monkeypatch.setattr(orch, "JOURNAL", journal)
265     return orch, signals, journal
266
267
268 def test_intraday_only_bars_do_not_create_equity_bias(tmp_path, monkeypatch):
269     import json
270
271     orch, signals, _journal = _pipeline_dirs(tmp_path, monkeypatch)
272     bars = _double_bottom_bars()
273     raw = {
274         "watchlists": [{"id": "1", "display_name": "T"}],
275         "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "AAPL"}]},
276         "fundamentals_by_symbol": {"AAPL": {"average_volume": 3_000_000}},
277         "historicals_by_symbol_timeframe": {"AAPL": {"10minute": bars}},
278         "buying_power": 1500.0,
279         "equity_quotes_by_symbol": {"AAPL": {"bid_price": "100.00", "ask_price": "100.10"}},
280         "equity_tradability_by_symbol": {"AAPL": {"regular_hours": {"buy": True}}},
281     }
282     summary = orch.run_pipeline(raw)
283     assert summary["equity_candidate_count"] == 0
284     payload = json.loads((signals / "equity_candidates.json").read_text())
285     reasons = {row["symbol"]: row["reason"] for row in payload["rejected"]}
286     assert reasons["AAPL"] == "equity_long_only_requires_bullish"
287
288
289 def test_option_candidate_uses_mid_and_falls_back_to_one_otm(tmp_path, monkeypatch):
290     import json
291     from datetime import timedelta
292
293     from pipeline.session import today_et
294
295     orch, signals, journal = _pipeline_dirs(tmp_path, monkeypatch)

```

```

296     bars = _double_bottom_bars()
297     exp = (today_et() + timedelta(days=3)).isoformat()
298     raw = {
299         "watchlists": [{"id": "1", "display_name": "T"}],
300         "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "AAPL"}]},
301         "fundamentals_by_symbol": {"AAPL": {"average_volume": 3_000_000}},
302         "historicals_by_symbol_timeframe": {"AAPL": {"day": bars}},
303         "buying_power": 1500.0,
304         "spots_by_symbol": {"AAPL": 100.0},
305         "option_chains_by_symbol": {"AAPL": {"expiration_dates": [exp]}},
306         "option_instruments_by_symbol_exp": {
307             f"AAPL{exp}": [
308                 {"id": "atm", "type": "call", "strike_price": "100"},
309                 {"id": "otm", "type": "call", "strike_price": "105"},
310                 {"id": "itm", "type": "call", "strike_price": "95"},
311             ]
312         },
313         "option_quotes_by_id": {
314             "atm": {
315                 "bid_price": "2.00",
316                 "ask_price": "2.10",
317                 "delta": "0.55",
318                 "mark_price": "9.99",
319             },
320             "otm": {
321                 "bid_price": "1.00",
322                 "ask_price": "1.05",
323                 "delta": "0.45",
324                 "mark_price": "9.99",
325             },
326         },
327         "equity_quotes_by_symbol": {"AAPL": {"bid_price": "100.00", "ask_price": "100.10"}},
328         "equity_tradability_by_symbol": {"AAPL": {"regular_hours": {"buy": True}}},
329     }
330     summary = orch.run_pipeline(raw)
331     assert summary["option_candidate_count"] == 1
332     assert summary["equity_candidate_count"] == 0
333     payload = json.loads((signals / "option_candidates.json").read_text())
334     cand = payload["candidates"][0]
335     assert cand["option_id"] == "otm"
336     assert cand["selection"] == "one_otm"
337     assert cand["premium_mark"] == pytest.approx(1.025)
338     assert cand["premium_source"] == "mid"
339     assert cand["cash_debit"] == pytest.approx(102.5)
340     assert payload["do_not_place"] is True
341     assert payload["h_entry_ready"] is False
342     assert "hour_confirm" in payload["h_still_requires"]
343     assert (journal / f"{today_et().isoformat()}.md").exists()
344
345
346 def test_inverse_etf_never_becomes_option_candidate(tmp_path, monkeypatch):
347     import json
348     from datetime import timedelta
349
350     from pipeline.session import today_et
351
352     orch, signals, _journal = _pipeline_dirs(tmp_path, monkeypatch)
353     bars = _double_bottom_bars()
354     exp = (today_et() + timedelta(days=3)).isoformat()
355     raw = {
356         "watchlists": [{"id": "1", "display_name": "T"}],
357         "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "SQQQ"}]},
358         "fundamentals_by_symbol": {
359             "SQQQ": {"average_volume": 20_000_000, "name": "ProShares UltraPro Short QQQ"}
360         },
361         "historicals_by_symbol_timeframe": {"SQQQ": {"day": bars}},
362         "buying_power": 1500.0,
363         "spots_by_symbol": {"SQQQ": 10.0},
364         "option_chains_by_symbol": {"SQQQ": {"expiration_dates": [exp]}},
365         "option_instruments_by_symbol_exp": {
366             f"SQQQ{exp}": [
367                 {"id": "atm", "type": "call", "strike_price": "10"},
368                 {"id": "otm", "type": "call", "strike_price": "11"},
369             ]
370         },

```

```

370         },
371         "option_quotes_by_id": {
372             "atm": {"bid_price": "1.00", "ask_price": "1.05", "delta": "0.45"},
373         },
374         "portfolio": {"start_of_day_equity": "1500.00", "total_value": "1512"},
375     }
376     summary = orch.run_pipeline(raw)
377     assert summary["option_candidate_count"] == 0
378     assert summary["equity_candidate_count"] == 0
379     assert "SQQQ" not in summary["eligible_equities"]
380     assert summary["bod_nlv"] == 1500.0
381     assert summary["bod_nlv_field"] == "start_of_day_equity"
382     universe = json.loads((signals / "universe.json").read_text())
383     reasons = {row["symbol"]: row["reason"] for row in universe["liquidity"]["rejected"]}
384     assert reasons["SQQQ"] == "inverse_etf"
385
386
387 def test_option_candidate_rejects_debit_above_buying_power(tmp_path, monkeypatch):
388     import json
389     from datetime import timedelta
390
391     from pipeline.session import today_et
392
393     orch, signals, _journal = _pipeline_dirs(tmp_path, monkeypatch)
394     bars = _double_bottom_bars()
395     exp = (today_et() + timedelta(days=3)).isoformat()
396     raw = {
397         "watchlists": [{"id": "1", "display_name": "T"}],
398         "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "AAPL"}]},
399         "fundamentals_by_symbol": {"AAPL": {"average_volume": 3_000_000}},
400         "historicals_by_symbol_timeframe": {"AAPL": {"day": bars}},
401         "buying_power": 1500.0,
402         "spots_by_symbol": {"AAPL": 100.0},
403         "option_chains_by_symbol": {"AAPL": {"expiration_dates": [exp]}},
404         "option_instruments_by_symbol_exp": {
405             f"AAPL[{exp}": [
406                 {"id": "atm", "type": "call", "strike_price": "100"},
407             ]
408         },
409         "option_quotes_by_id": {
410             "atm": {"bid_price": "19.50", "ask_price": "20.50", "delta": "0.45"},
411         },
412         "equity_quotes_by_symbol": {"AAPL": {"bid_price": "100.00", "ask_price": "100.10"}},
413         "equity_tradability_by_symbol": {"AAPL": {"regular_hours": {"buy": True}}},
414     }
415     summary = orch.run_pipeline(raw)
416     assert summary["option_candidate_count"] == 0
417     payload = json.loads((signals / "option_candidates.json").read_text())
418     reasons = {row["symbol"]: row["reason"] for row in payload["equity_fallbacks"]}
419     assert reasons["AAPL"] == "exceeds_buying_power"
420
421
422 def test_incomplete_option_page_is_rejected(tmp_path, monkeypatch):
423     import json
424     from datetime import timedelta
425
426     from pipeline.session import today_et
427
428     orch, signals, _journal = _pipeline_dirs(tmp_path, monkeypatch)
429     bars = _double_bottom_bars()
430     exp = (today_et() + timedelta(days=3)).isoformat()
431     raw = {
432         "watchlists": [{"id": "1", "display_name": "T"}],
433         "watchlist_items_by_id": {"1": [{"object_type": "instrument", "symbol": "AAPL"}]},
434         "fundamentals_by_symbol": {"AAPL": {"average_volume": 3_000_000}},
435         "historicals_by_symbol_timeframe": {"AAPL": {"day": bars}},
436         "buying_power": 1500.0,
437         "spots_by_symbol": {"AAPL": 200.0},
438         "option_chains_by_symbol": {"AAPL": {"expiration_dates": [exp]}},
439         "option_instruments_by_symbol_exp": {
440             f"AAPL[{exp}": [{"id": "far", "type": "call", "strike_price": "100"}]
441         },
442         "option_quotes_by_id": {
443             "far": {"bid_price": "1.00", "ask_price": "1.05", "delta": "0.45"},

```

```
444         },
445     }
446     summary = orch.run_pipeline(raw)
447     assert summary["option_candidate_count"] == 0
448     payload = json.loads((signals / "option_candidates.json").read_text())
449     reasons = {row["symbol"]: row["reason"] for row in payload["equity_fallbacks"]}
450     assert reasons["AAPL"] == "option_chain_incomplete_atm_not_in_page"
```

```
1 from datetime import datetime
2 from zoneinfo import ZoneInfo
3
4 from pipeline.execution import (
5     build_option_entry_proposal,
6     can_place_live,
7     load_latest_equity_candidates,
8     load_latest_option_candidates,
9 )
10
11 ET = ZoneInfo("America/New_York")
12 SUNDAY = datetime(2026, 8, 30, 12, 0, tzinfo=ET)
13 MONDAY_RTH = datetime(2026, 8, 31, 10, 0, tzinfo=ET)
14
15
16 def test_place_blocked_when_playbook_draft():
17     ok, reason = can_place_live(
18         explicit_confirm=True, playbook_released=False, h_enabled=False, now=SUNDAY
19     )
20     assert not ok
21     assert reason == "options_playbook_still_draft"
22
23
24 def test_place_blocked_without_confirm():
25     ok, reason = can_place_live(
26         explicit_confirm=False, playbook_released=True, h_enabled=False, now=SUNDAY
27     )
28     assert not ok
29     assert reason == "missing_explicit_user_confirm"
30
31
32 def test_place_blocked_outside_rth_even_with_confirm():
33     ok, reason = can_place_live(
34         explicit_confirm=True, playbook_released=True, h_enabled=True, now=SUNDAY
35     )
36     assert not ok
37     assert reason == "outside_rth"
38
39
40 def test_place_blocked_after_1545_even_if_h_disabled():
41     monday_1545 = datetime(2026, 8, 31, 15, 45, tzinfo=ET)
42     ok, reason = can_place_live(
43         explicit_confirm=True, playbook_released=True, h_enabled=False, now=monday_1545
44     )
45     assert not ok
46     assert reason == "no_new_entries_after_1545"
47
48
49 def test_place_blocked_while_h_owns_rth():
50     ok, reason = can_place_live(
51         explicit_confirm=True, playbook_released=True, h_enabled=True, now=MONDAY_RTH
52     )
53     assert not ok
54     assert reason == "h_owns_rth_while_enabled"
55
56
57 def test_place_blocked_before_0945_for_options_even_if_h_disabled():
58     monday_0930 = datetime(2026, 8, 31, 9, 30, tzinfo=ET)
59     monday_0944 = datetime(2026, 8, 31, 9, 44, 59, tzinfo=ET)
60     monday_0945 = datetime(2026, 8, 31, 9, 45, tzinfo=ET)
61     blocked, reason = can_place_live(
62         explicit_confirm=True, playbook_released=True, h_enabled=False, now=monday_0930
63     )
64     assert not blocked and reason == "no_new_option_entries_before_0945"
65     blocked2, reason2 = can_place_live(
66         explicit_confirm=True, playbook_released=True, h_enabled=False, now=monday_0944
67     )
68     assert not blocked2 and reason2 == "no_new_option_entries_before_0945"
69     ok, ok_reason = can_place_live(
70         explicit_confirm=True, playbook_released=True, h_enabled=False, now=monday_0945
71     )
72     assert ok and ok_reason is None
73
74
```

```
75 def test_equity_place_allowed_at_open_if_h_disabled():
76     monday_0930 = datetime(2026, 8, 31, 9, 30, tzinfo=ET)
77     ok, reason = can_place_live(
78         explicit_confirm=True,
79         playbook_released=True,
80         playbook_kind="equity",
81         h_enabled=False,
82         now=monday_0930,
83     )
84     assert ok and reason is None
85
86
87 def test_place_allowed_during_rth_if_h_disabled():
88     ok, reason = can_place_live(
89         explicit_confirm=True, playbook_released=True, h_enabled=False, now=MONDAY_RTH
90     )
91     assert ok and reason is None
92
93
94 def test_place_allowed_during_rth_with_override():
95     ok, reason = can_place_live(
96         explicit_confirm=True,
97         playbook_released=True,
98         h_enabled=True,
99         now=MONDAY_RTH,
100         h_rth_override=True,
101     )
102     assert ok and reason is None
103
104
105 def test_proposal_is_one_contract_buy_to_open():
106     cand = {
107         "symbol": "SOFI",
108         "structure": "long_put",
109         "expiration": "2026-09-04",
110         "option_type": "put",
111         "strike": "18.0",
112         "option_id": "abc",
113         "playbook_status": "DRAFT_NOT_RELEASED",
114     }
115     p = build_option_entry_proposal(cand, limit_price="0.43")
116     assert p["quantity"] == "1"
117     assert p["places_order"] is False
118     assert p["legs"][0]["side"] == "buy"
119     assert p["legs"][0]["position_effect"] == "open"
120
121
122 def test_load_latest_skips_historical_do_not_place(tmp_path, monkeypatch):
123     import json
124
125     import pipeline.execution as execution
126
127     monkeypatch.setattr(execution, "SIGNALS", tmp_path)
128     (tmp_path / "option_candidates.json").write_text(
129         json.dumps(
130             {
131                 "historical": True,
132                 "do_not_place": True,
133                 "candidates": [{"symbol": "SOFI"}],
134             }
135         )
136     )
137     (tmp_path / "equity_candidates.json").write_text(
138         json.dumps({"do_not_place": True, "candidates": [{"symbol": "AAPL"}]})
139     )
140     assert load_latest_option_candidates() == []
141     assert load_latest_equity_candidates() == []
142
143     (tmp_path / "option_candidates.json").write_text(
144         json.dumps({"candidates": [{"symbol": "NU"}]})
145     )
146     assert load_latest_option_candidates()[0]["symbol"] == "NU"
```